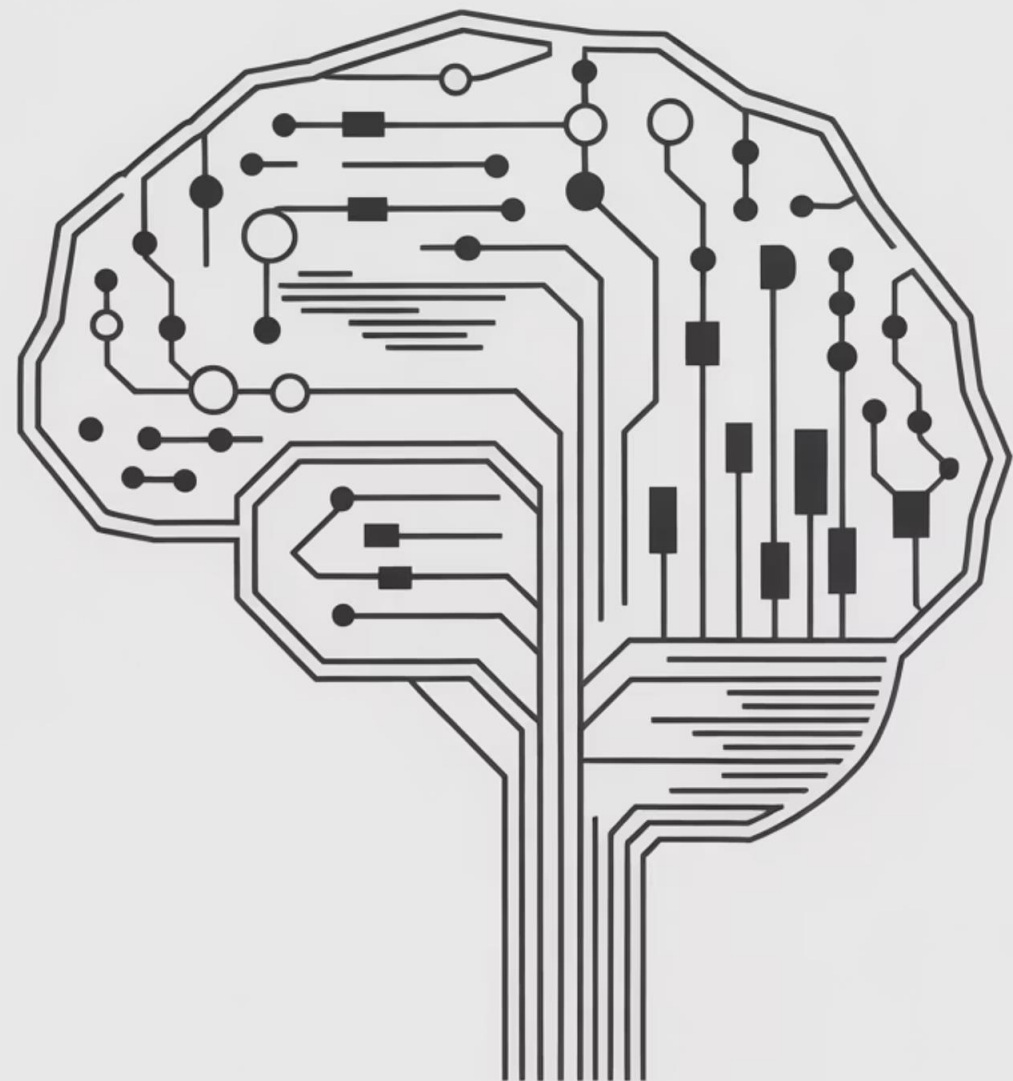


Современные языковые модели: от токенов к агентам

Комплексное путешествие в мир
современного искусственного интеллекта
для первокурсников



Как я смотрю на LLM и прочие абстракции

$$y = f(x)$$

Ожидаемый
ответ

МО / LLM /
«AI»

Данные

Общение с веб-версией

X – Промпт

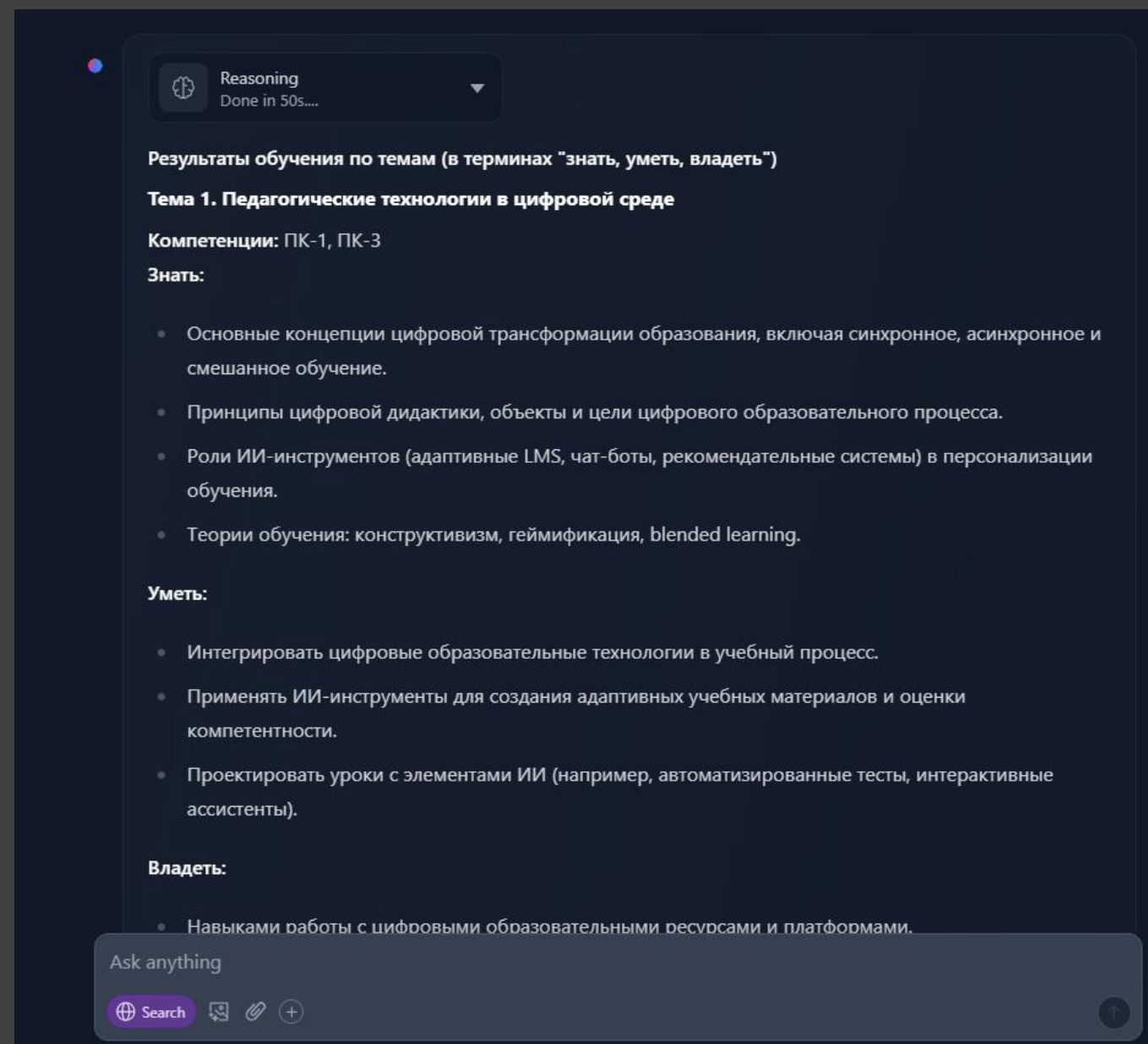
Описание задачи с необходимым контекстом: темы РПД и перечень компетенций из паспорта компетенций

y – Результаты

Результаты обучения в терминах «знать, уметь, владеть»

f – Модель

Одна из LLM доступных в сервисе chat от huggingface (Qwen3-235B-A22B)



Cursor

X – Промпт

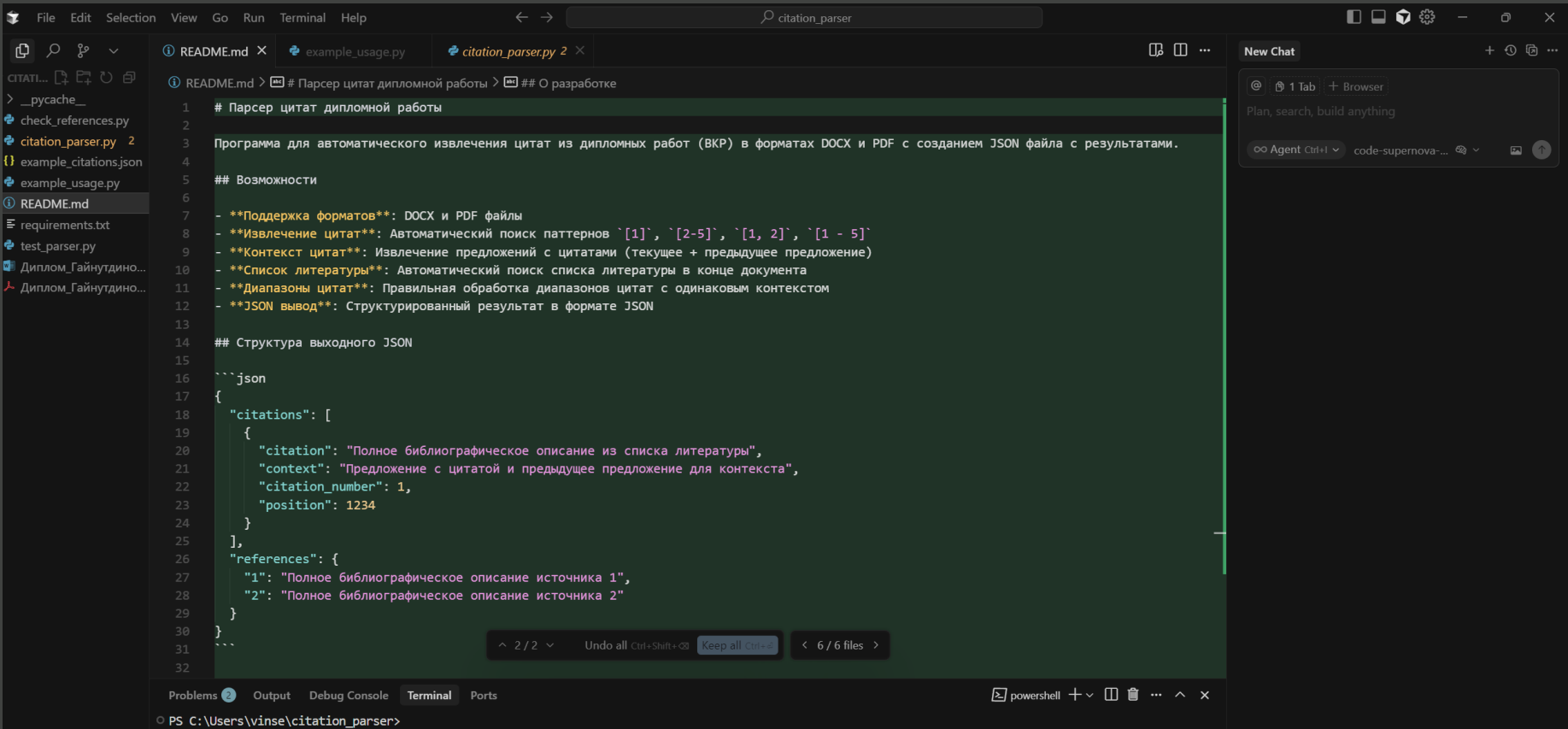
Описание того какой код я хочу
получить и для чего на естественном
языке

y – Результаты

Готовые скрипты на разных языках
программирования

f – Модель

Cursor IDE, разные модели «под
копотом»



«Простые» системы на базе LLM

X – Эссе

по определенной тематике

у – Результаты

оценка эссе в «числовом» формате, а также комментарии по каждому из пунктов

f – Модель и Системный Промпт

Одна из LLM небольшого размера, системный промпт в виде развернутого рубрикатора оценок

Essay Evaluation with LLM Agent

Enter your essay here:

Фильм "Аватар" Джеймса Кэмерона представляет собой яркий пример дискурса коэволюции Природы и Общества, где центральным посылом является идея гармоничного сосуществования, а не покорения или противостояния. Аргументы на примере коэволюционного дискурса:

Evaluate Essay

Evaluation Results:

disclosure score: 5

disclosure feedback: Эссе демонстрирует глубокое понимание экологического дискурса и его различных аспектов. Автор анализирует фильм "Аватар" как пример дискурса коэволюции Природы и Общества, подчеркивая важность гармоничного сосуществования человека и природы. Эссе включает в себя детальный анализ различных аспектов дискурса, включая экологическую этику, эпистемологическое измерение и контраргументы дискурса покорения природы. Автор также демонстрирует способность критически мыслить и анализировать сложные концепции.

Evaluation Results:

Total score: 29/30

Russian score: 5/5

originality score: 5

originality feedback: Эссе демонстрирует сильную, уникальную точку зрения и творческое мышление. Автор предлагает независимые идеи и избегает чрезмерной зависимости от общепринятых точек зрения.

argumentation score: 5

argumentation feedback: Эссе демонстрирует глубокое понимание концепции коэволюции Природы и Общества и ее применения в контексте фильма «Аватар». Автор приводит убедительные аргументы, включая примеры из фильма, которые подчеркивают важность гармоничного сосуществования человека и природы. Эссе также демонстрирует способность критически мыслить о дискурсе покорения природы и его противоположности.

structure score: 4

structure feedback: Эссе демонстрирует четкую структуру с введением, основной частью и заключением. Логическая последовательность в основном последовательна, но некоторые переходы между частями могут быть резкими или недостаточно развитыми.

clarity score: 5

clarity feedback: Эссе демонстрирует четкое понимание концепции коэволюции Природы и Общества и ее применения в контексте фильма «Аватар». Автор эффективно использует аргументы и контраргументы, чтобы продемонстрировать преимущества дискурса коэволюции над дискурсом покорения природы. Текст написан четко и доступно, что делает его понятным для аудитории.

grammatic score: 5

grammatic feedback: Эссе демонстрирует четкое понимание концепции коэволюции Природы и Общества и ее применения в контексте фильма «Аватар». Автор обеспечивает детальные аргументы и примеры, которые поддерживают дискурс коэволюции.

«Сложные» системы на базе LLM

X – Пожелания студентов

направление магистратуры, навыки и
хотелки магистранта

y – Результаты

карточка с темой ВКР и доп
информацией

f – Агенты

Агентная система на базе моделей с
открытым доступом через openreuter

Помощник выбора темы

привет

Здравствуйте! Рад помочь вам с выбором темы для магистерской диссертации по программе **09.04.01 Инженерия искусственного интеллекта**. Вижу, что ваши интересы лежат на стыке **Машинного обучения (МО), Медицинской диагностики и Биомедицинских исследований**. Это очень перспективное и востребованное направление!

Чтобы сэкономить ваше время, я сразу предложу несколько начальных направлений, основанных на ваших интересах. Выберите то, что вам кажется наиболее привлекательным, или скажите, если ни одно из них не подходит:

Предварительные темы (ИИ в Медицине и Биомедицине):

- Глубокое обучение для анализа медицинских изображений:**
Разработка сверточных нейронных сетей (CNN) для автоматического обнаружения и классификации патологий (например, опухолей на МРТ/КТ).

Введите сообщение...

Сгенерировать темы

Отправить

Idea

Создать тему

Создано

Отправлено

29.09.2025, 12:24

Применение генеративно-сопоставительных сетей для аугментации данных в задачах ранней диагностики редких патологий: Аспект повышения робастности классификаторов

Исследование посвящено разработке и валидации методологии аугментации биомедицинских данных с использованием генеративно-сопоставительных сетей (GAN) для решения проблемы дисбаланса классов и малой выборки, характерной для редких патологий. Акцент делается на повышении робастности и обобщающей способности нейросетевых классификаторов в условиях ограниченной и зашумленной реальной выборки, что критически важно для надежной ранней диагностики. Результаты исследования внесут вклад в развитие прикладного ИИ в медицинской сфере.

Открыть

30.09.2025, 16:24

Применение методов SHAP и LIME для объяснения решений нейросетевых классификаторов в задаче прогнозирования исходов сердечно-сосудистых заболеваний на основе мультимодальных биомедицинских данных

Исследование посвящено разработке

29.09.2025, 04:32

Разработка и валидация глубоких сверточных нейронных сетей для автоматизированной сегментации опухолевых образований в мультимодальных медицинских изображениях

Разработка и валидация глубоких сверточных нейронных сетей для автоматизированной сегментации опухолевых образований в мультимодальных медицинских изображениях

Открыть

29.09.2025, 05:33

Применение трансформеров и самоконтролируемого обучения для выявления биомаркеров и прогнозирования исходов в биомедицинских временных рядах

Применение трансформеров и самоконтролируемого обучения для выявления биомаркеров и прогнозирования исходов в биомедицинских временных рядах

Открыть

29.09.2025, 12:20

Применение трансферного

Содержание презентации

1

Token is All You Need

Основы токенизации текста

2

Embedding is All You Need

Векторные представления

3

Attention is All You Need

Механизм внимания

4

Instruction is All You Need

Следование инструкциям

5

RAG is All You Need

Расширенная генерация

6

Agent is All You Need

Агентные системы



Почему это важно для вас?

Понимание работы языковых моделей — это не просто теория. Это ключ к успешной карьере в области искусственного интеллекта, который революционизирует все сферы жизни от медицины до образования.

Эти знания станут вашим фундаментом для создания собственных AI-решений и понимания того, как работают системы, которые уже изменяют мир вокруг нас.

Token is All You Need

Что такое токен?

Представьте, что вы хотите научить компьютер понимать текст. С чего начать? Компьютер не понимает слова так, как мы. Ему нужны базовые строительные блоки.

Токен — это минимальная единица текста, которую обрабатывает языковая модель. Это может быть слово, часть слова или даже символ. Токены для языковой модели — это то же самое, что молекулы для вещества: базовые элементы, из которых строится всё остальное.

Почему не просто слова?

Вы можете спросить: "Почему бы не использовать просто слова?" Проблема в том, что в любом языке существуют миллионы возможных слов, особенно если учитывать разные формы, опечатки и новые термины.

Токенизация решает эту проблему, находя баланс между гибкостью и эффективностью. Частые слова остаются целыми токенами, а редкие разбиваются на известные части.

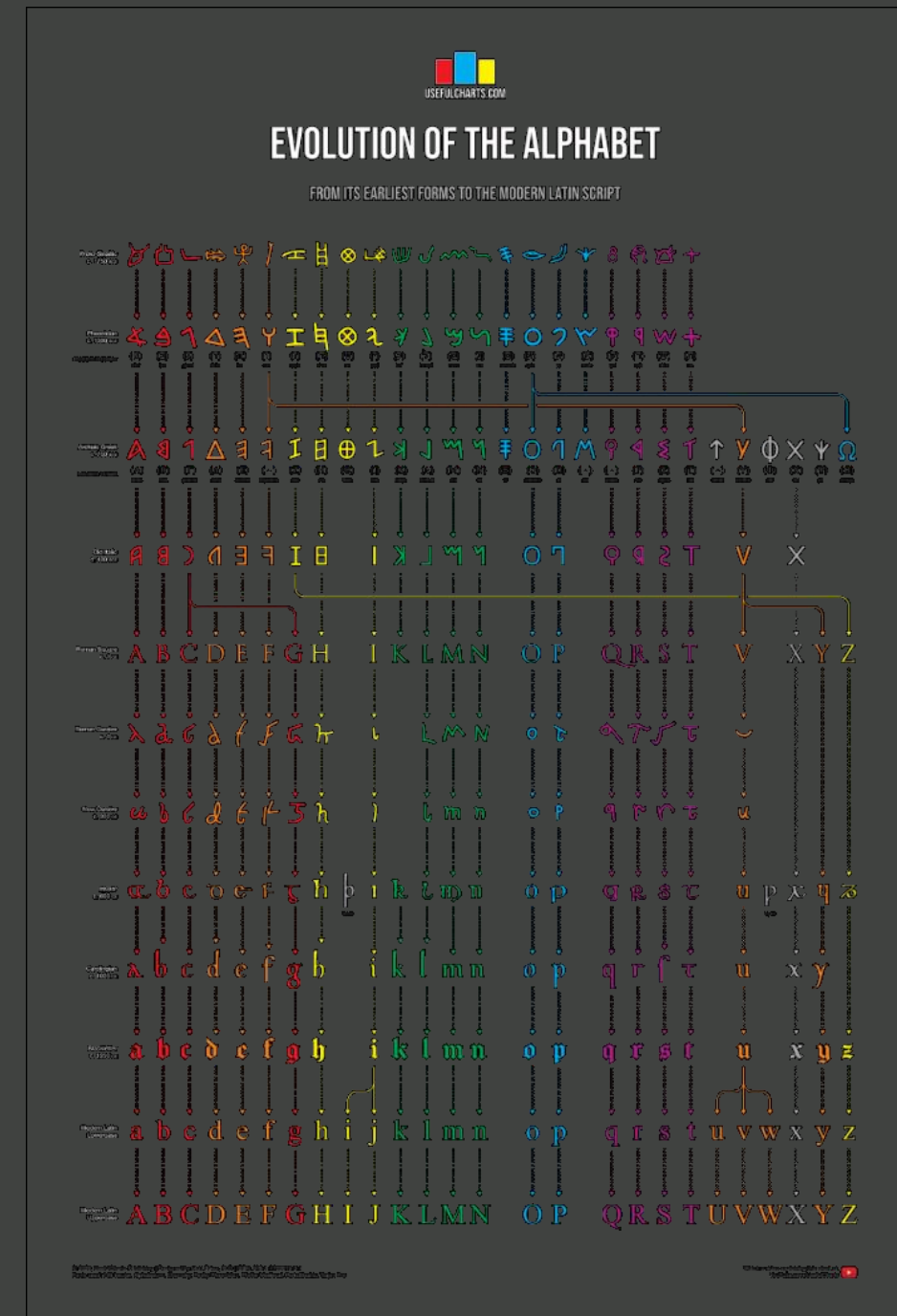
Про токенизацию

Египетские символы

- 1 токен = 1 слово
- Смысл в каждом токене
- Очень большой алфавит

Латинский алфавит

- 1 токен = 1 буква
- «Смысл» в хитром объединении токенов
- Алфавит небольшой



Методы токенизации

1

По словам

Самый простой подход: "Я учу машинное обучение" → ["Я", "учу", "машинное", "обучение"]

Проблема: огромный словарь и неизвестные слова

2

По подсловам

Разбиение на значимые части: "машинное" → ["машин", "ное"]

Преимущество: обработка новых слов через известные части

3

По символам

Каждый символ — отдельный токен: "кот" → ["к", "о", "т"]

Проблема: потеря смысловых связей и очень длинные последовательности

4

BPE (Byte-Pair Encoding)

Золотая середина: начинаем с символов, объединяем частые пары

Результат: оптимальный баланс между размером словаря и гибкостью

Byte-Pair Encoding (BPE)

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

BPE — это алгоритм сжатия данных, который итеративно заменяет наиболее частые пары байтов в последовательности на один неиспользованный байт.

Алфавит (Букв)

Начинаем с базового набора символов

("h" "u" "g", 10), ("p" "u" "g", 5), ("p" "u" "n", 12),
("b" "u" "n", 4), ("h" "u" "g" "s", 5)

Самая частая пара

Находим наиболее частую комбинацию символов

("h" "ug", 10), ("p" "ug", 5), ("p" "u" "n", 12),
("b" "u" "n", 4), ("h" "ug" "s", 5)

Итерации

Повторяем процесс до достижения нужного размера словаря

Словарь (Слов)

Получаем финальный набор токенов

("hug", 10), ("p" "ug", 5), ("p" "un", 12),
("b" "un", 4), ("hug" "s", 5)

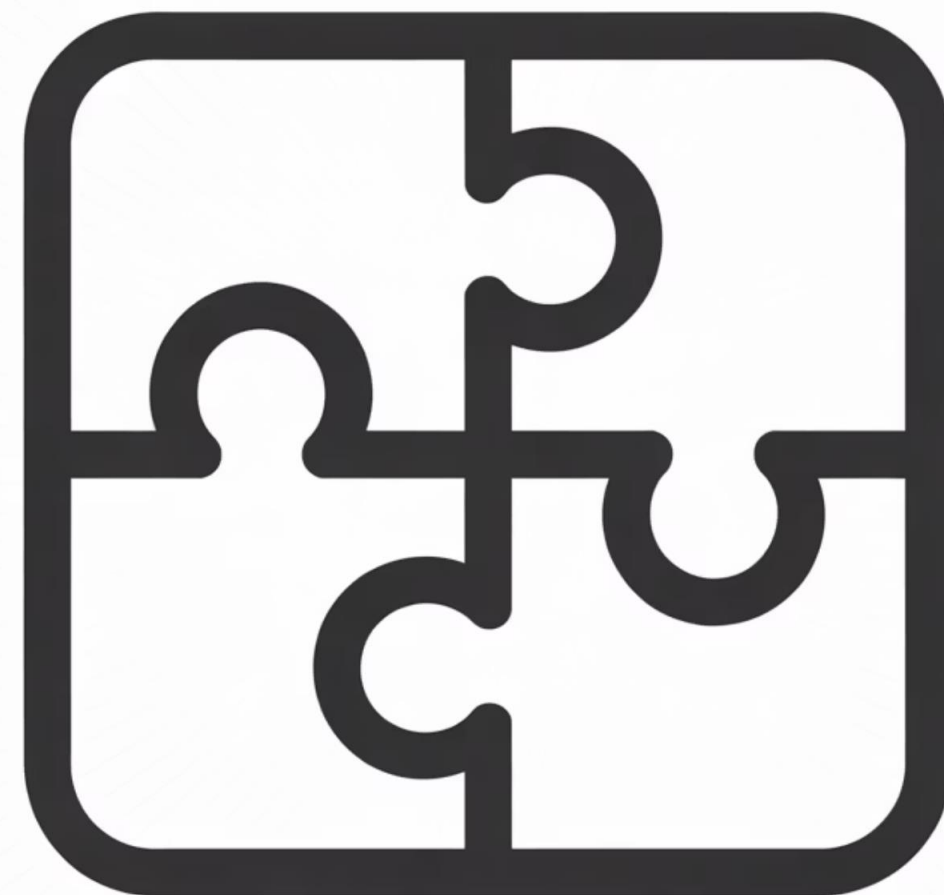
Преимущества ВРЕ

Эффективность для частых слов

Слова, которые встречаются постоянно (например, "обучение", "модель", "данные"), становятся отдельными токенами. Это ускоряет обработку самых распространённых случаев.

Гибкость для редких слов

Редкие или новые слова (например, "антидискриминационный") разбиваются на известные части ("анти", "дискриминац", "ионный"). Модель может их понять через составные части.



Токен — сколько в словах?

... В среднем один токен состоит из 3-4 символов.

Количество токенов в промпте с контекстом и ответе зависит от модели....

<https://developers.sber.ru/docs/ru/gigachat/limitations>

❏ **Правило для английского: 1 токен $\approx$ 4 символа $\approx$ $\frac{3}{4}$ слова. 100 токенов $\approx$ 75 слов.**

<https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them>

Примеры разных токенизаций

1801, 1825, 1855, 1881, 1894, 1917

Модель	Примеры токенов
ai-forever/ruGPT-3.5-13B	лицсят взгля никак место K
openai/whisper-large-v3	ón nothing friends
suno/bark	▽⊗◆、【け
FacebookAI/xlm-roberta-base	♡telahنىnikか\lại

https://tiktokenizer.vercel.app/?model=cl100k_base

Практическое значение токенизации

Понимание токенизации критически важно для работы с языковыми моделями, потому что:



Стоимость запросов

API языковых моделей тарифицируются по токенам, а не по словам или символам. Знание токенизации помогает оптимизировать расходы.



Ограничения контекста

Модели имеют лимит на количество токенов в контексте (например, 4096 или 32000). Понимание токенизации помогает эффективно использовать это окно.



Различия между языками

Разные языки токенизируются по-разному. Русский текст обычно требует больше токенов, чем английский той же длины.

Токенизация: резюме

X – Вход

Набор слов, чисел, специальных символов, включая смайлики

f – Функция

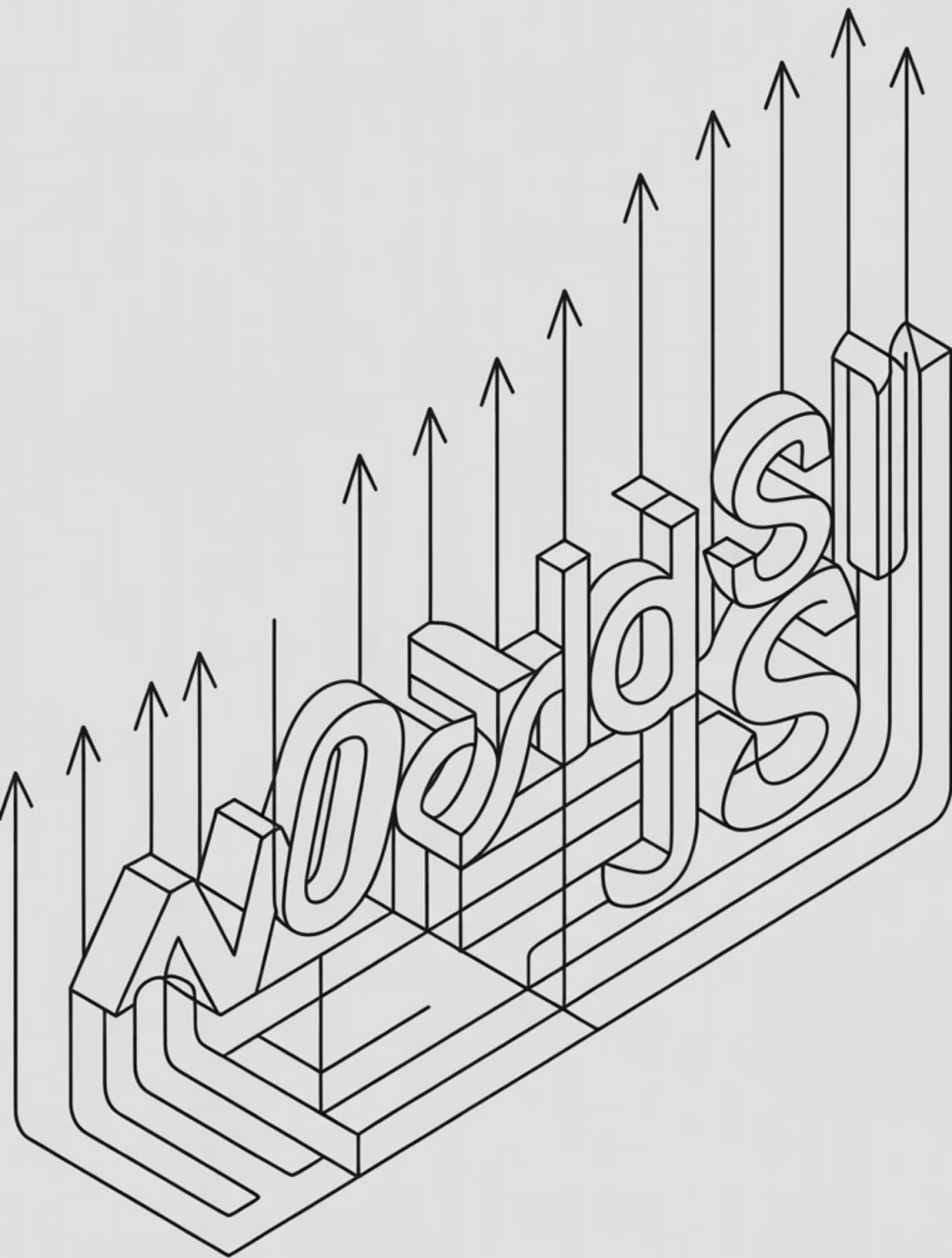
Токенизатор:
BytePairEncoding,
WordPieceEncoding или
SentencePieceEncoding

y – Выход

Последовательность
порядковых номеров
токенов в «словаре»
модели



Embedding is All You Need



От токенов к числам

После того как текст разбит на токены, возникает следующая задача: как представить эти токены в форме, понятной для математических вычислений?

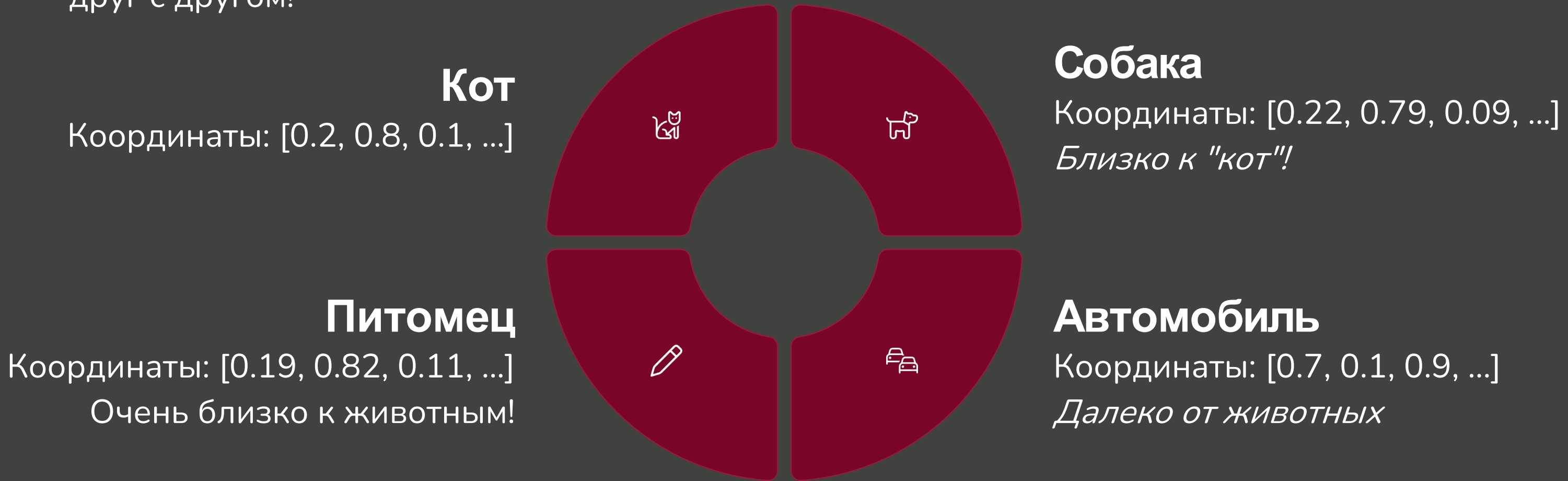
Здесь на сцену выходят **эмбединги** — векторные представления токенов.

Эмбединг — это список чисел (обычно от 300 до 16 384*), который кодирует **значение и контекст слова**.

Можно представить это как координаты слова в многомерном пространстве смыслов.

Волшебство векторного пространства

Представьте огромное многомерное пространство, где каждое слово имеет свое место. Причем слова располагаются не случайно — похожие по смыслу слова находятся рядом друг с другом!



Проблема 1: Семантика

Прилагательное:

имеющий цвет растущей листвы;
заросший зеленью, растительностью;
перен. техн. жарг. комп. жарг. Экологичный;
перен. незрелый, неспелый
перен. молодой, неопытный, плохо знающий жизнь

Глагол:

увеличиваться в размерах
перен. воспитываться
перен. совершенствоваться

Зеленая трава **растет** быстро

Существительное:

растение, не образующее одревесневающих стеблей;
только ед. ч.: покров земли из таких растений;
жарг. любое наркотическое вещество,
предназначенное для курения

Наречие:

нареч. к быстрый;
через небольшой промежуток
времени

Проблема 2: Позиция слова

1. You have completed the task **well**. [наречие]
2. You are doing **well** in machine learning. [прилагательное]
3. **Well**, not everyone are participating equally. [междометие]
4. The **well** is empty. [существительное]
5. Tears are starting to **well** in his eyes. [глагол]

Слово «**well**» меняет значение в зависимости от контекста

Проблема 3: Грамматические связи

Собака не пошла по дороге, потому что **она мокрая**
(дорога мокрая)

Собака не пошла по дороге, потому что **она боится**
(собака боится)

Модель должна понимать, к чему относится местоимение «**она**» в каждом случае.



Как создаются эмбединги?

Эмбединги не придумываются вручную — они обучаются автоматически на огромных объемах текста.

Процесс обучения основан на простом принципе: слова, которые встречаются в похожих контекстах, должны иметь похожие эмбединги.

"You shall know a word by the company it keeps"

— Firth, J. R. 1957:11



Как создаются эмбединги?

Контекстное обучение

Модель анализирует миллионы предложений:

- "Кот спит на диване"
- "Собака спит в будке"
- "Автомобиль стоит в гараже"

Она замечает, что "кот" и "собака" встречаются в похожих контекстах (оба спят, оба в определенных местах), поэтому их эмбединги становятся похожими.

Многомерность смысла

Каждое измерение вектора кодирует определенный аспект значения:

- Одушевленность/неодушевленность
- Размер объекта
- Позитивность/негативность
- Абстрактность/конкретность

Хотя интерпретация не всегда очевидна

Векторная магия: король - мужчина + женщина = ?

Одно из самых удивительных свойств эмбедингов — возможность выполнять *семантическую арифметику*.

$$\text{вектор("король")} - \text{вектор("мужчина")} + \text{вектор("женщина")} \approx \text{вектор("королева")}$$

Это работает, потому что разница между "король" и "мужчина" кодирует понятие королевского статуса, которое можно "перенести" на женщину, получив королеву.

Другие примеры:

- Москва - Россия + Франция $\approx$ Париж
- Быстрый - быстро + медленный $\approx$ медленно
- Хороший - лучший + плохой $\approx$ худший

Gensim: примеры работы

```
word_vectors.most_similar( positive=['Russia', 'White_House'], negative=['Kremlin'], topn=5)
('Oval_Office', 0.553)
('United_States', 0.547)
('Obama', 0.540)
('President_Barack_Obama', 0.535)
('Bush', 0.534)
```

```
word_vectors.most_similar( positive=['Vodka', 'Japan'], negative=['Russia'], topn=5)
('Shochu', 0.553)
('shochu', 0.522)
('Momokawa', 0.521)
('saké', 0.520)
('Sadaharu', 0.514)
```

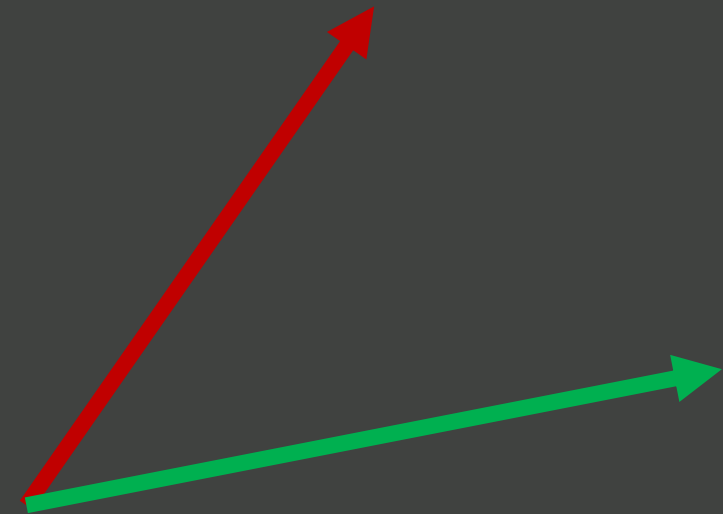
```
word_vectors.most_similar(positive=['balalaika', 'Hawaii'], negative=['Russia'], topn=5)
('ukulele', 0.572)
('hula_halau', 0.543)
('Makaha_Sons', 0.540)
('ukulele_virtuoso_Jake_Shimabukuro', 0.534)
('kumu', 0.526)
```

Математика сходства

Как измерить, насколько два слова близки по смыслу? Используется косинусное расстояние между их векторами-эмбедингами.

Формула косинусного сходства:

$$\text{Cosine Distance} = \frac{a \cdot b}{|a||b|}$$



Результат от -1 до 1, где:

- 1 — векторы направлены одинаково (максимальное сходство)
- 0 — векторы перпендикулярны (нет сходства)
- -1 — векторы направлены противоположно (противоположные значения)

Размерность эмбедингов

Типичные размеры эмбедингов в современных моделях:

300

Word2Vec

Классический метод,
популярный в 2013-2017 годах

768

BERT-base

Стандарт для многих задач
обработки текста

1536

GPT-3

Большие модели используют
больше измерений для точности

4096

GPT-4

Современные крупные модели с
максимальной точностью

Больше измерений = более тонкое различение нюансов значения, но и больше вычислительных затрат.

Вложения / Embedding: резюме

X – Вход

Последовательность порядковых номеров токенов в «словаре» модели

f – Функция

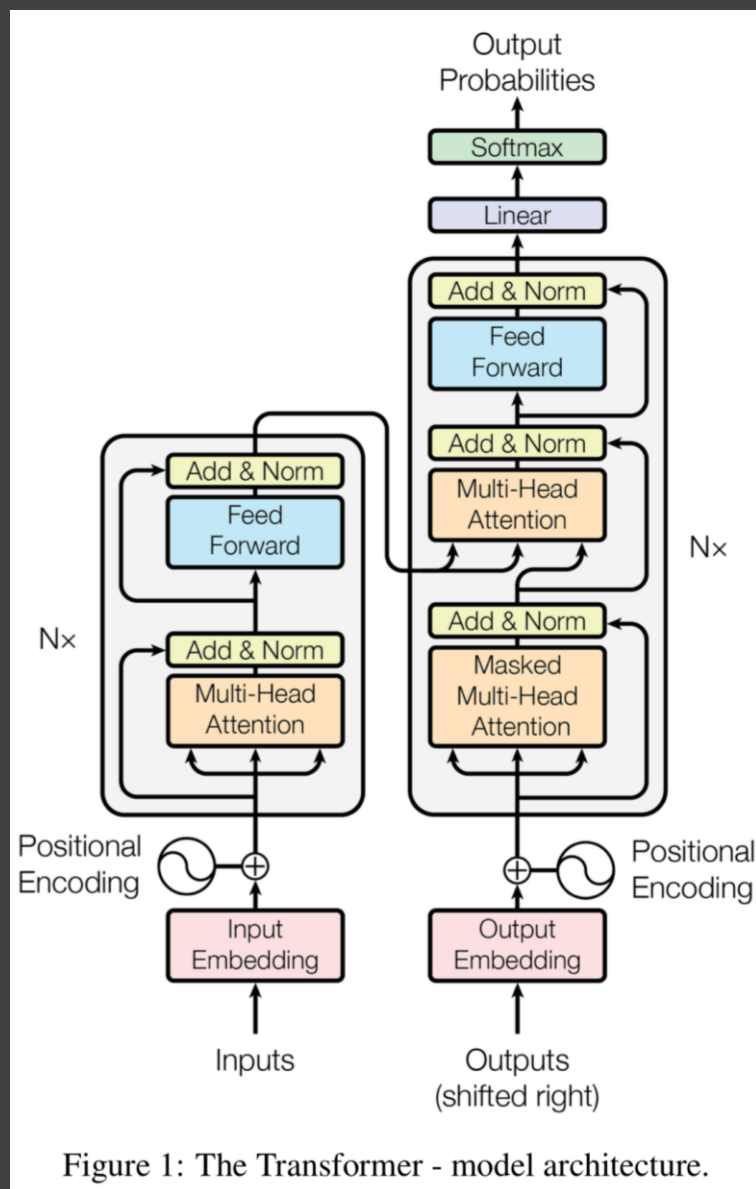
Таблица поиска (lookup table), специфичная для каждой модели

y – Выход

Последовательность векторов, специфичных для модели

Attention is All You Need

Transformer Encoder Decoder



X – Вход

слова на
одном
естественн
ом языке

f – Функция

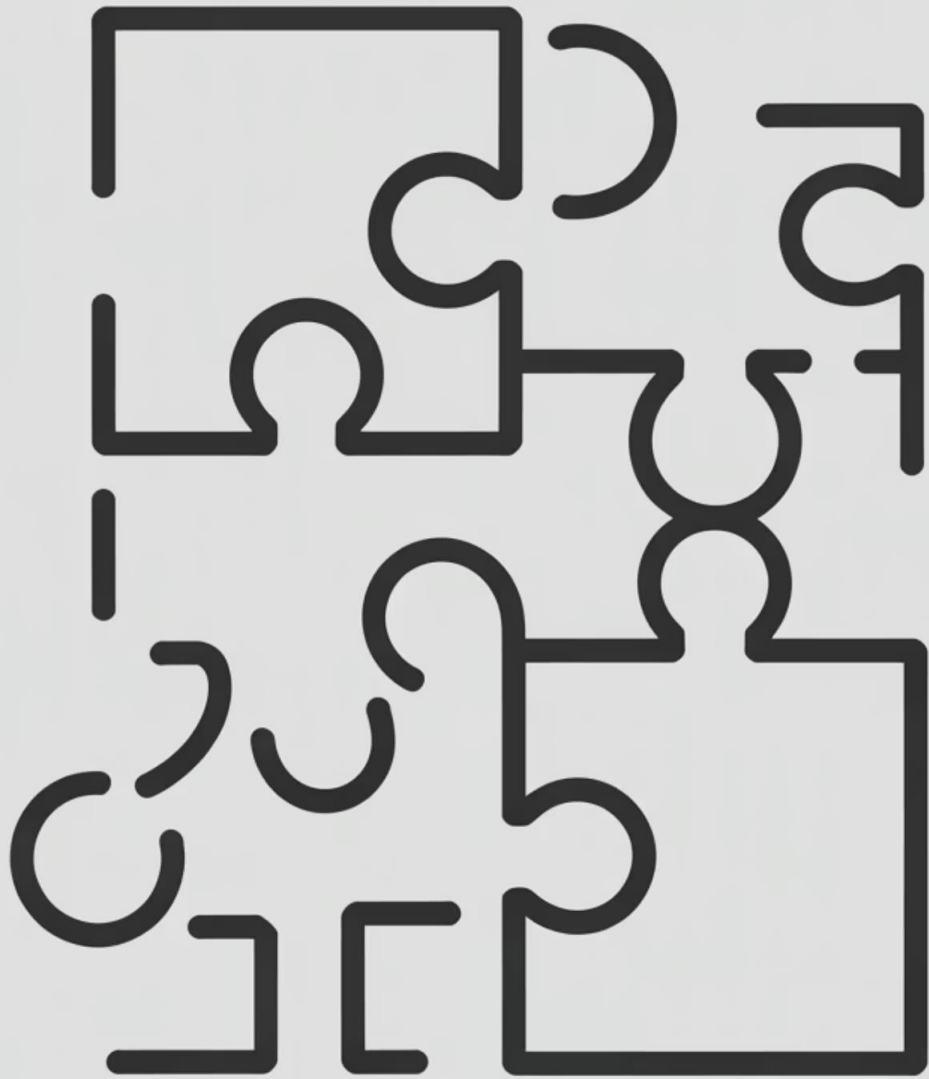
много
матричных
умножений,
которые нам
надо разобрать

y – Выход

слова на
другом
естественном
языке

Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

<https://arxiv.org/pdf/1706.03762.pdf>



Проблема: как понять связи между словами?

Представьте предложение: "Кот, который жил у соседа через дорогу, любил спать на моём коврике". Когда мы читаем слово "любил", мы понимаем, что это действие относится к коту, хотя между ними много других слов.

Как научить модель понимать эти связи? Простой обработки слов по порядку недостаточно — нужен механизм, который может **обращать внимание** на важные слова независимо от их позиции.

Attention: революция в обработке языка

Механизм внимания (Attention) — это ключевая инновация, которая лежит в основе всех современных языковых моделей (Transformer, BERT, GPT). Он позволяет модели динамически определять, какие части входного текста важны при обработке каждого конкретного слова.

До Attention (RNN)

- Обработка слов строго последовательно
- Информация "затухает" на длинных дистанциях
- Сложно учитывать дальние связи
- Медленное обучение (нельзя распараллелить)

С Attention (Transformer)

- Каждое слово "смотрит" на все остальные одновременно
- Дальние связи учитываются так же хорошо, как близкие
- Модель сама определяет важность связей
- Быстрое параллельное обучение



Аналогия с детективом

Представьте детектива, расследующего дело. У него есть:

Вопросы (Query, Q)

"Что я ищу?" — конкретная информация, которую нужно выяснить в данный момент

Улики (Key, K)

"Что у меня есть?" — доступная информация, которую можно проверить

Значимость улики (Value, V)

"Насколько это важно?" — релевантность каждой улики для текущего вопроса

Детектив сравнивает свои вопросы с уликами и определяет, какие улики наиболее релевантны. Точно так же работает Attention!

Математика Attention

Механизм внимания вычисляется по следующей формуле:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1

Шаг 1: Вычисление сходства

QK^T — матричное произведение запросов и ключей.

Результат показывает, насколько каждый запрос "совпадает" с каждым ключом.

2

Шаг 2: Масштабирование

Делим на $\sqrt{d_k}$ (корень из размерности), чтобы «стабилизировать градиенты при обучении».

3

Шаг 3: Нормализация

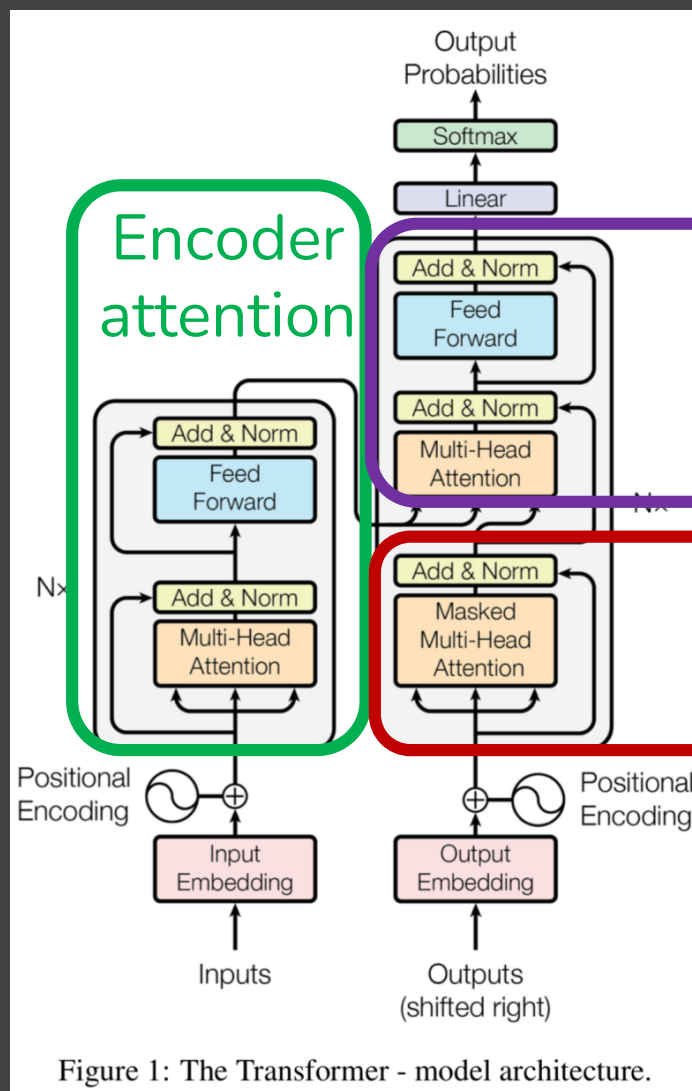
Softmax превращает числа в вероятности (от 0 до 1), которые в сумме дают 1. Это и есть веса внимания.

4

Шаг 4: Взвешенная сумма

Умножаем веса на значения V — получаем итоговое представление, где важные элементы имеют больший вес.

Разные типы внимания



- **Encoder attention**
Связь между словами в исходном языке
- **Decoder attention**
Связь между генерируемыми словами в языке на который переводят
- **Encoder-decoder attention**
Связь между разными языками

Способ моделировать связь между конкретными словами

<https://arxiv.org/pdf/1706.03762.pdf>

Multi-Head Attention: несколько точек зрения

Современные модели используют не один, а несколько механизмов внимания параллельно — это называется **Multi-Head Attention**. Каждая "голова" фокусируется на разных аспектах связей.

Голова 1: Синтаксис

Фокусируется на грамматических связях: подлежащее-сказуемое, определение-определяемое.

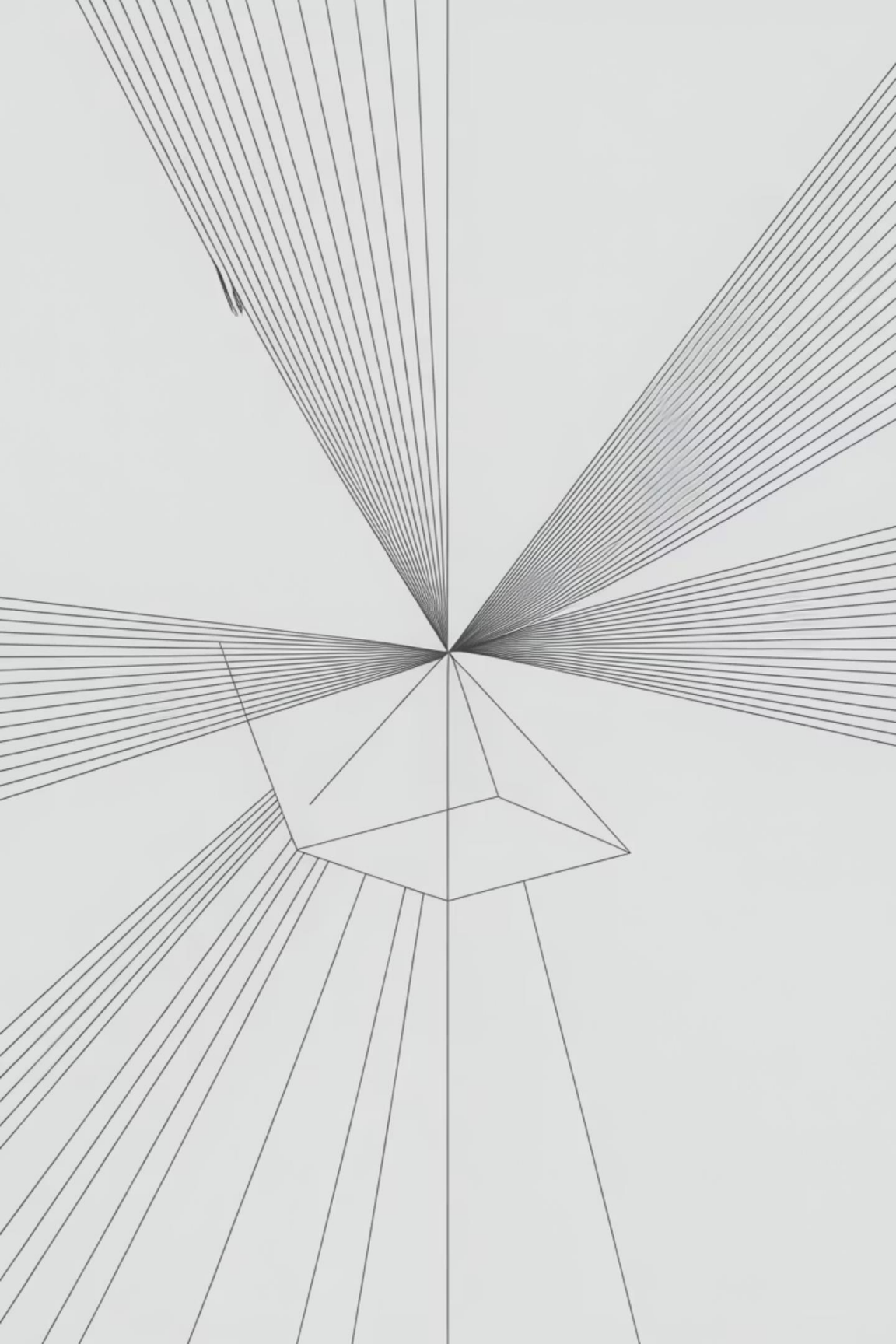
Голова 2: Семантика

Ищет смысловые связи между концепциями и идеями в предложении.

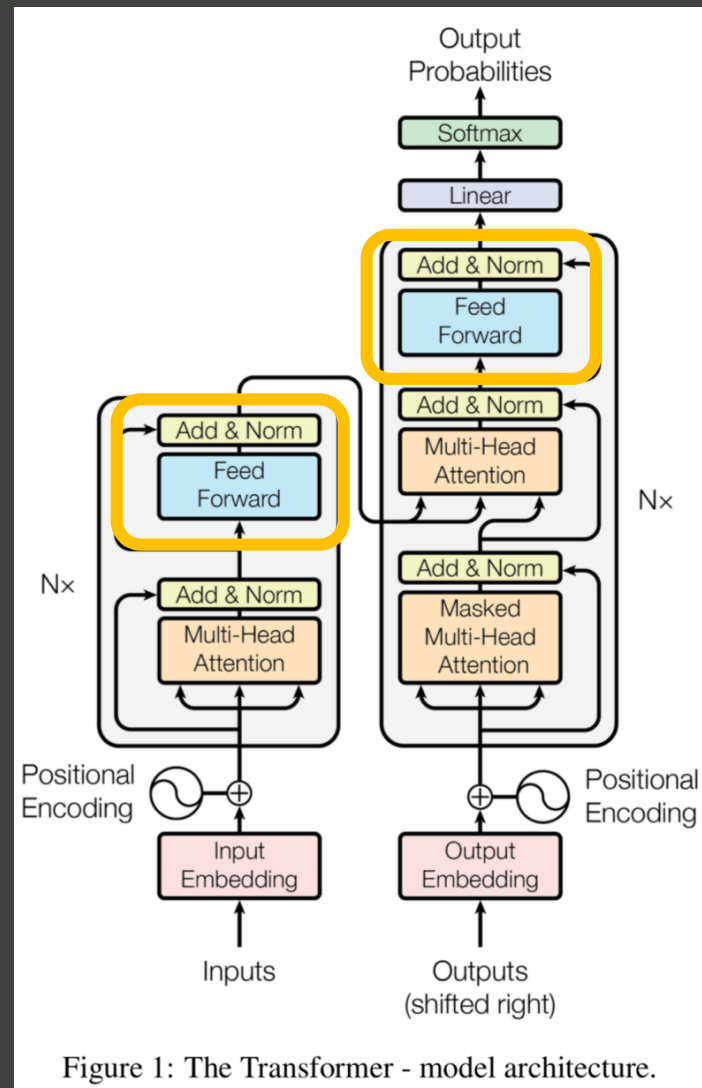
Голова 3: Кореференция

Определяет, какие местоимения к каким объектам относятся.

Обычно используется несколько голов внимания, которые затем объединяются для получения богатого многоаспектного представления.



Feed Forward



«Простая» полносвязная сеть
Multi-layer Perceptron

$$FFN(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$$

Способ хранить статичные факты о языках и
совмещать их с взаимодействием слов

Transformer Encoder Decoder

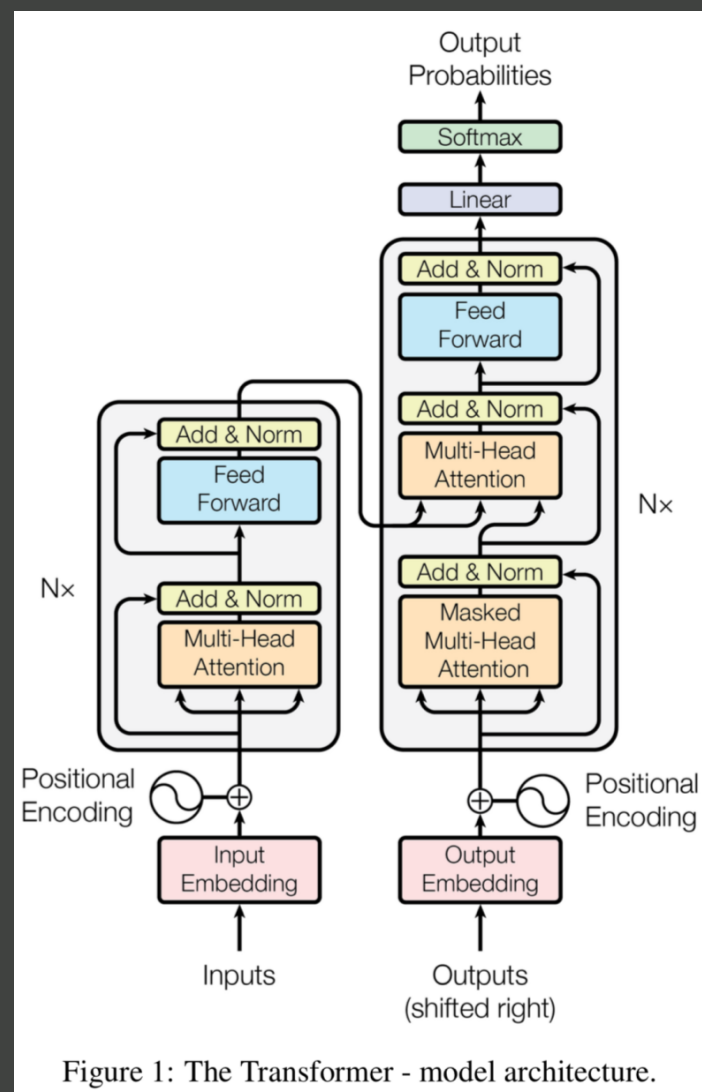


Figure 1: The Transformer - model architecture.

‘Классическая’ архитектура

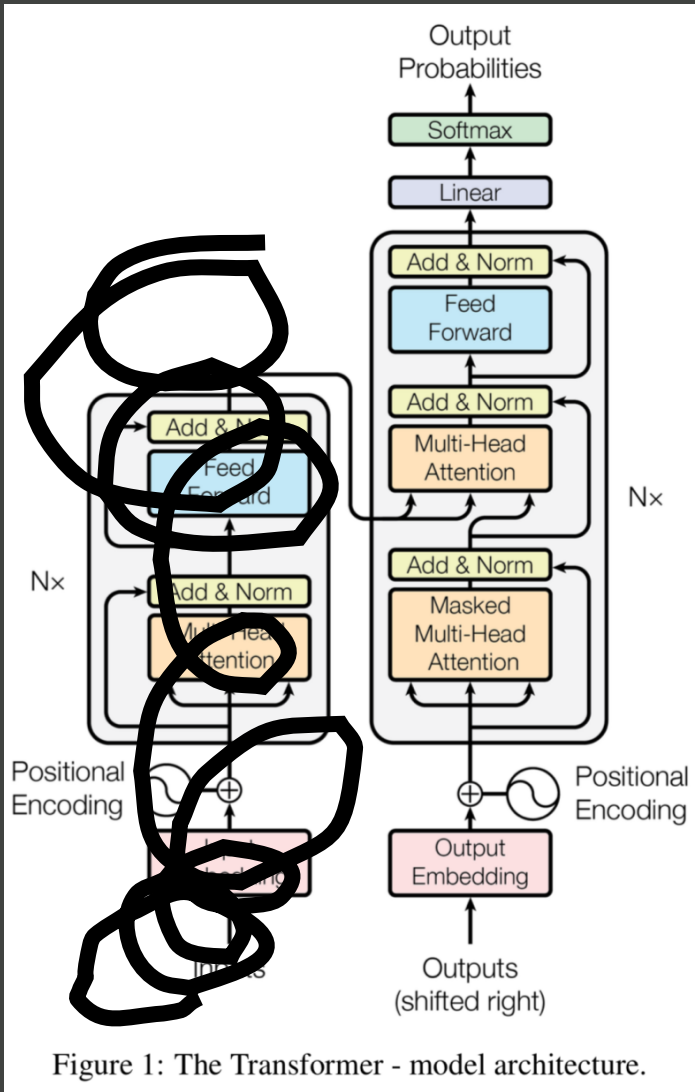
65 млн. обучаемых параметров / весов

Применение Google Translate

Все веса, и из внимания (W_n^Q , W_n^K , W_n^V , W^n),
и из Feed Forward (W_1 , b_1 , W_2 , b_2) едины для всех входов

<https://arxiv.org/pdf/1706.03762.pdf>

GPT - Transformer Decoder

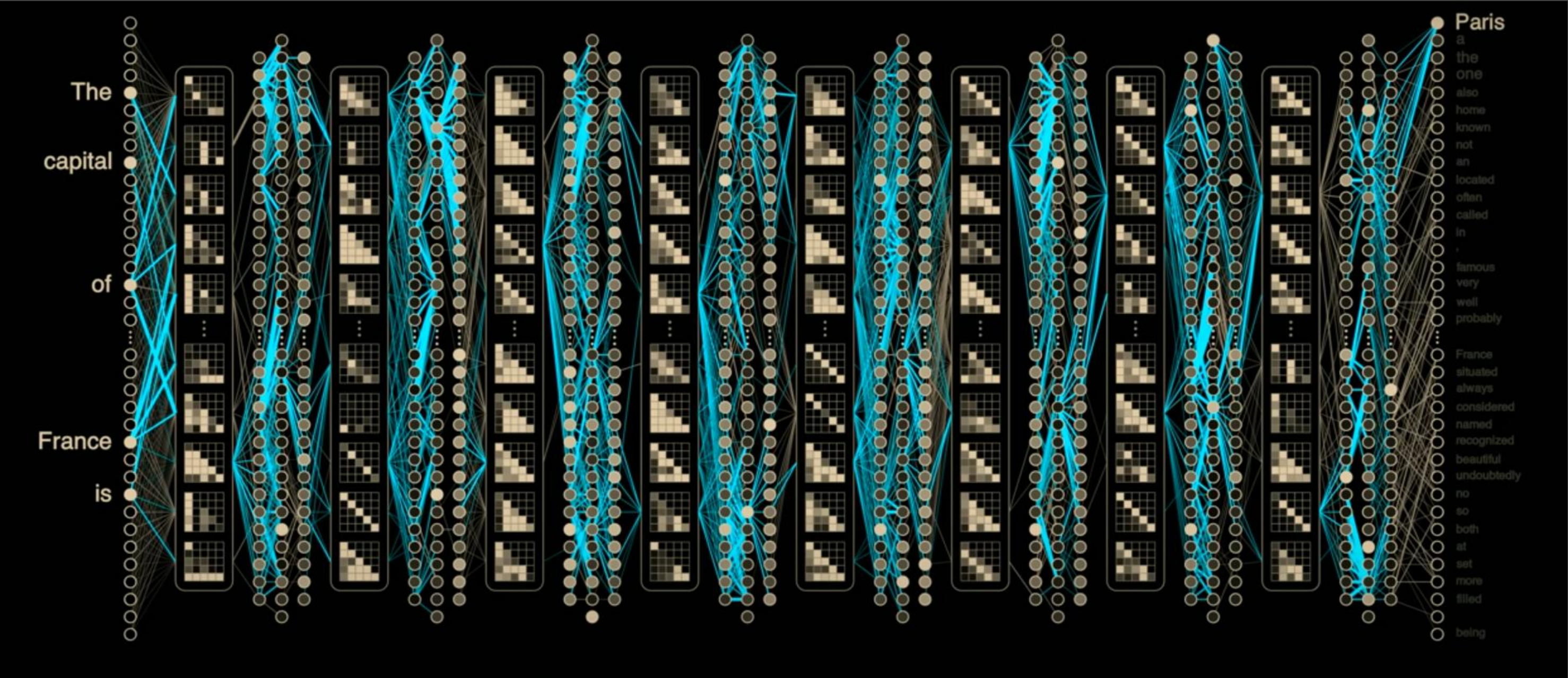


Generative Pre-Training

«...просто предсказывает следующее слово...»

Год	Модель	Млрд параметров	Контекст, токены	Размер данных, Гб
2018	GPT	0.117	512	4
2019	GPT-2	0.345/0.762/1.542	1 024	40
2020	GPT-3	175	2 048	570
2023	GPT-4	?? (1 760)	8 192 / 32 768	????

Наглядно



<https://youtu.be/VkHfRKewkWw?si=RHWTqVGchuvvc53G1>

Предсказание следующего слова (токена, ну вы поняли)

Что может модель, которая просто предсказывает следующее слово?

Запрос	Ожидаемый ответ	Тип задачи
$2 + 3 \times 2 =$	8	Математика
Столица Франции –	Париж	География / Знания
2 4 6 8	10	Логика / Последовательность
Синоним: "большой" –	огромный	Лексика / Словарь
Переведи: "Hello" –	Привет	Перевод

Markdown — структурированный текст

Markdown — простой способ создавать структурированный текст: заголовки, списки, таблицы, код — всё это модель генерирует как последовательность токенов.

```
markdown
1 # 🚀 Эволюция языковых моделей
2
3 ## 📖 Что такое LLM?
4
5 Большие языковые модели (LLM) учатся **предсказывать следующий токен** — слово, символ или часть слова.
6 Но их возможности — гораздо шире, чем просто "дописать предложение" 😊
7
8 ## 🧠 Что они могут помимо текста?
9
10 ### ✅ Форматирование
11 Модели легко генерируют Markdown с:
12 - Заголовками разных уровней ( `#` , `##` , `###` )
13 - Списками (как этот!)
14 - **Жирным**, *курсивом*, `кодом`
15
16 ### 🎨 Визуализация через текст
17 Они "рисуют" с помощью:
18 - ASCII-арт 🖼️
19 - Диаграмм Mermaid 📊
20 - Таблиц 📄
21
22 ### ✂️ Пример таблицы
23
24 | Поколение | Пример | Особенность |
25 |-----|-----|-----|
26 | GPT-1 | 2018 | Классификация |
27 | GPT-3 | 2020 | Few-shot обучение |
28 | GPT-4 | 2023 | Мультимодальность 🗨️ 🖼️ |
29
30 ## 🏁 Вывод
31
32 > Модели не просто "говорят" — они **структурируют**, **форматируют** и даже **визуализируют** знания! 🤖
```

Эволюция языковых моделей

Что такое LLM?

Большие языковые модели (LLM) учатся **предсказывать следующий токен** — слово, символ или часть слова. Но их возможности — гораздо шире, чем просто "дописать предложение" 😊

Что они могут помимо текста?

Форматирование

Модели легко генерируют Markdown с:

- Заголовками разных уровней (`#` , `##` , `###`)
- Списками (как этот!)
- Жирным, курсивом, кодом

Визуализация через текст

Они "рисуют" с помощью:

- ASCII-арт 🖼️
- Диаграмм Mermaid 📊
- Таблиц 📄

Пример таблицы

Поколение	Пример	Особенность
GPT-1	2018	Классификация
GPT-3	2020	Few-shot обучение
GPT-4	2023	Мультимодальность 🗨️ 🖼️

Вывод

Модели не просто "говорят" — они **структурируют**, **форматируют** и даже **визуализируют** знания! 🤖

LaTeX — математические формулы

LaTeX — ещё один яркий пример того, как языковые модели могут генерировать не просто текст, а структурированный, семантически насыщенный код, который превращается в формулы, документы и даже математические доказательства.

```
1 P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}
```

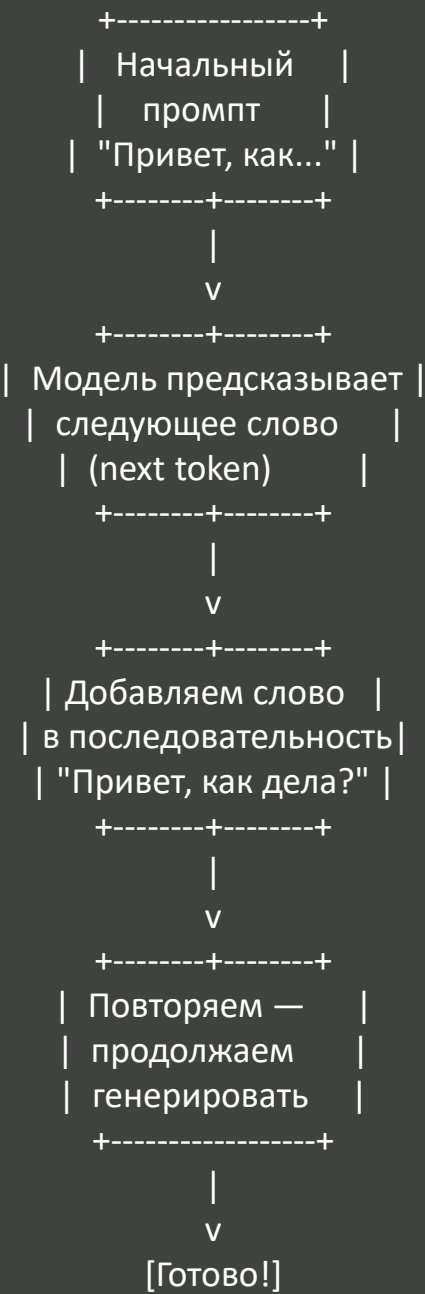
$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Пример: Небольшой LaTeX-документ

```
latex
1 \documentclass{article}
2 \usepackage[utf8]{inputenc}
3 \title{\textbf{Умные модели и текст}}
4 \author{AI Лекция 2025}
5 \date{}
6
7 \begin{document}
8
9 \maketitle
10
11 \section{Как работают LLM?}
12 Языковые модели предсказывают следующий токен.
13 Но они могут генерировать не только слова – \textit{а даже \LaTeX}!
14
15 \subsection{Пример формулы}
16 Вероятность события по Байесу:
17 \[
18 P(\theta|D) = \frac{P(D|\theta) P(\theta)}{P(D)}
19 \]
20
21 \end{document}
```


ASCII-арт — рисование символами

Из символов можно строить изображения, диаграммы и интерфейсы. Модель учится предсказывать нужные символы, чтобы «нарисовать» что-то осмысленное.



Mermaid — диаграммы в коде

Mermaid — язык для описания диаграмм (графов, последовательностей, блок-схем) в текстовом виде. Модель генерирует код Mermaid, который потом превращается в визуализацию.

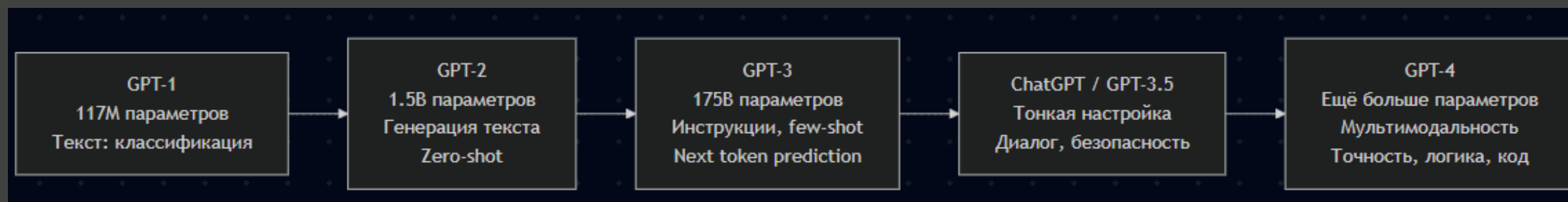
```
graph LR
```

```
  A["GPT-1<br>117M параметров<br>Текст: классификация"] --> B["GPT-2<br>1.5B параметров<br>Генерация текста<br>Zero-shot"]
```

```
  B --> C["GPT-3<br>175B параметров<br>Инструкции, few-shot<br>Next token prediction"]
```

```
  C --> D["ChatGPT / GPT-3.5<br>Тонкая настройка<br>Диалог, безопасность"]
```

```
  D --> E["GPT-4<br>Ещё больше параметров<br>Мультимодальность<br>Точность, логика, код"]
```



Главное: модель не «видит» изображение — она предсказывает символы так, будто пишет программу, которая создаст изображение или форматированный контент.

Контекст постоянно меняется



Про контекст

Модель	Контекстное окно (токены)	Год выхода
GPT-2	1 024	2019
GPT-3.5	4 096 (до 16 384)	2022
GPT-4	8 192 (до 32 768)	2023
Mixtral 8x7B	32 768	2023
LLaMA-3	8 192	2024
Qwen (включая Qwen-1.5/2)	32 768 (до 131 072)	2023–2024

Внутри каждого трансформера

1

Вход

Последовательность слов

2

Токены

Превращаются в Embedding

3

Обработка

Механизм внимания

4

Выход

Вероятности токенов



Генеративные модели: резюме

X – Вход

Естественный язык,
превращенный в токены

f – Функция

Векторы Embedding,
взаимодействующие через
механизм внимания с
«базой фактов» из
интернета

y – Выход

Последовательность
вероятностей токенов

Умная говорилка?

Диапазон знаний его был грандиозен. Ни одной сказки и ни одной песни он не знал больше чем наполовину, но зато это были русские, украинские, западнославянские, немецкие, английские, по-моему, даже японские, китайские и африканские сказки, легенды, притчи, баллады, песни, романсы, частушки и припевки.

Склероз приводил его в бешенство... Но единственной песенкой, которую он допел до конца, был «Чижик-пыжик», а единственной сказочкой, которую он связно рассказал, был «Дом, который построил Джек» в переводе Маршака, да и то с некоторыми купюрами.

— Аркадий и Борис Стругацкие, «Понедельник начинается в субботу»



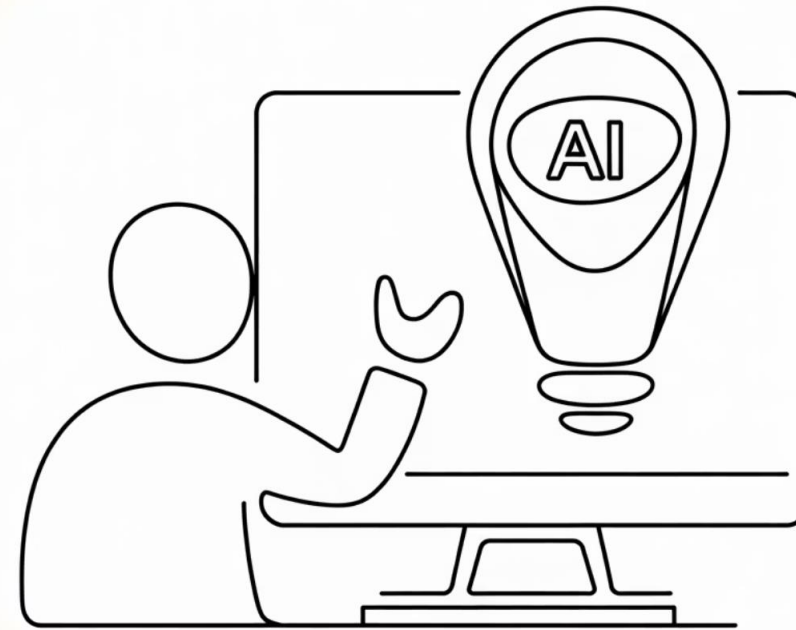
Instruction is All You Need



От предсказателя к помощнику

Базовая языковая модель обучена на одной простой задаче: **предсказывать следующий токен**. Если дать ей текст "Кот сидит на", она продолжит "коврике" или другим подходящим словом.

Но для практического применения нам нужна модель, которая умеет следовать инструкциям: отвечать на вопросы, писать код, переводить тексты, суммировать документы.



Сложные запросы

Модели могут выполнять разнообразные задачи:



Ответы на вопросы

Какой самый большой океан на Земле? →
Тихий океан



Генерация текста

Напиши короткий рассказ о путешествии в
горы



Создание контента

Напиши пост в блог о преимуществах
чтения книг



Создание диалогов

Напиши диалог между двумя друзьями,
которые планируют поездку



Создание резюме

Напиши резюме для позиции менеджера
проекта

Что такое Instruction Tuning?

Instruction tuning — это процесс дообучения модели на примерах выполнения конкретных инструкций. Модель учится понимать, что от неё хотят, и генерировать соответствующие ответы.

Формат обучающих данных

Инструкция: "Переведи на английский язык"

Входные данные: "Привет, как дела?"

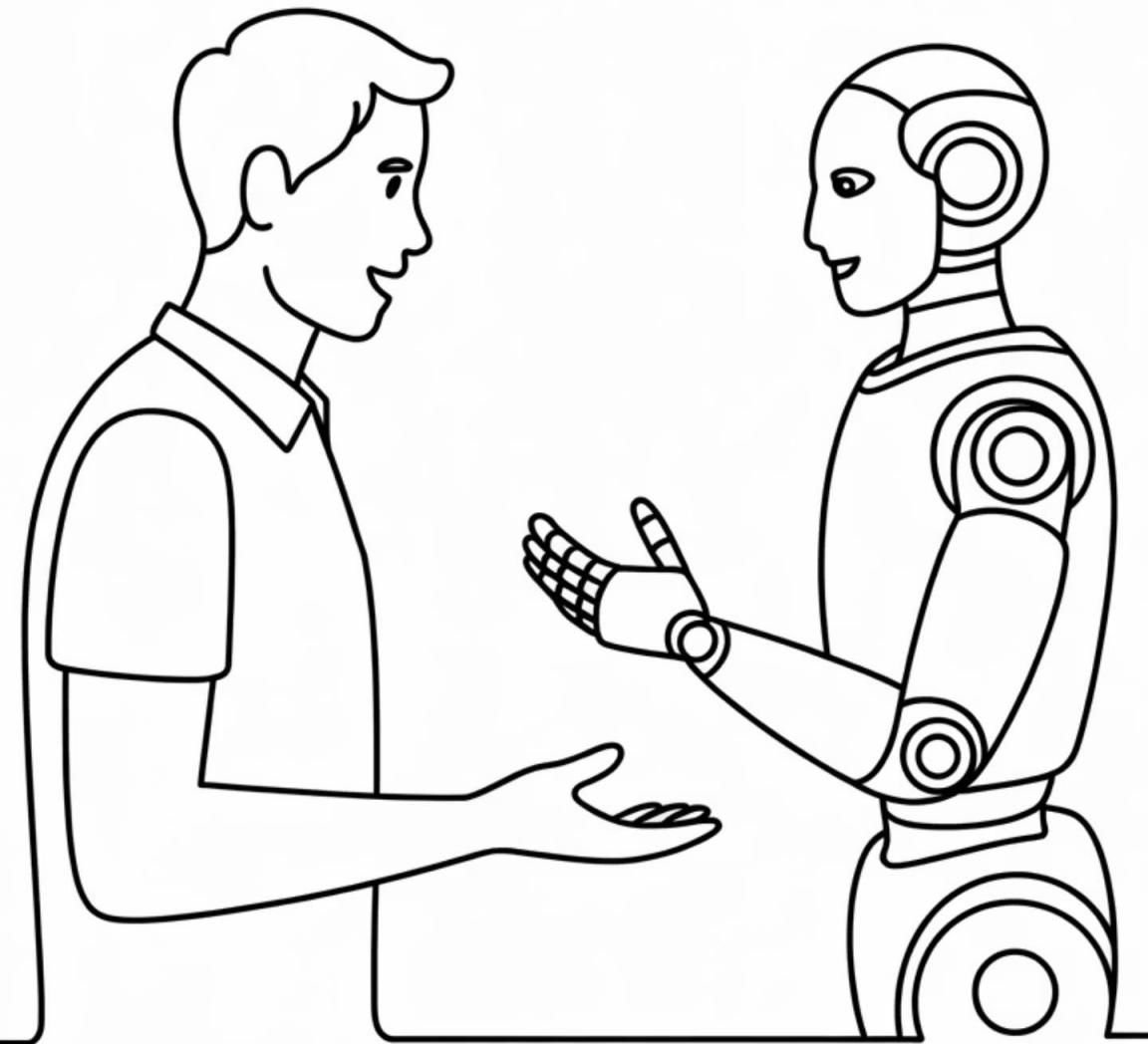
Ожидаемый ответ: "Hello, how are you?"

Разнообразие задач

Модель обучается на тысячах различных типов инструкций: ответы на вопросы, классификация, суммаризация, генерация текста, анализ тональности, решение математических задач и многое другое.

Обобщение на новые задачи

После обучения модель может выполнять инструкции, которые не встречались в обучающих данных, благодаря пониманию общего паттерна "инструкция → действие".



RLHF: обучение от человеческой обратной связи

Reinforcement Learning from Human Feedback (RLHF) — продвинутая техника, где модель учится не только правильным ответам, но и предпочтениям людей.

01

Сбор сравнений

Люди оценивают несколько вариантов ответов модели на один запрос:
"Какой ответ лучше?"

02

Обучение reward model

На основе человеческих оценок обучается модель вознаграждения, которая предсказывает, насколько ответ понравится людям.

03

Оптимизация через RL

Языковая модель дообучается методами обучения с подкреплением, максимизируя вознаграждение от reward model.

04

Итерации

Процесс повторяется: модель улучшается, генерирует новые ответы, получает новые оценки.

Именно RLHF сделал ChatGPT таким полезным и "человечным" в общении. yyyyyyyyyyyyyyy

Как оценить ответ: демография оценщиков ChatGPT

Гендер

- Мужчины: 50.0%
- Женщины: 44.4%
- Небинарные: 5.6%

Возраст

- 18-24: 26.3%
- 25-34: 47.4%
- 35-44: 10.5%

Национальность

- Филиппинцы: 22%
- Бангладешцы: 22%
- Американцы: 17%
- Другие: 39%

Образование

- Бакалавриат: 52.6%
- Магистратура: 36.8%
- Школа: 10.5%

Chain-of-Thought: учим модель думать

Одна из самых мощных техник instruction tuning — Chain-of-Thought (CoT), или "цепочка рассуждений". Вместо прямого ответа, модель учится объяснять свои действия пошагово.

Без Chain-of-Thought

Вопрос: Посчитай сумму 17, 23 и 18

Ответ: 58

Результат может быть правильным, но процесс скрыт. При ошибке непонятно, где именно она возникла.

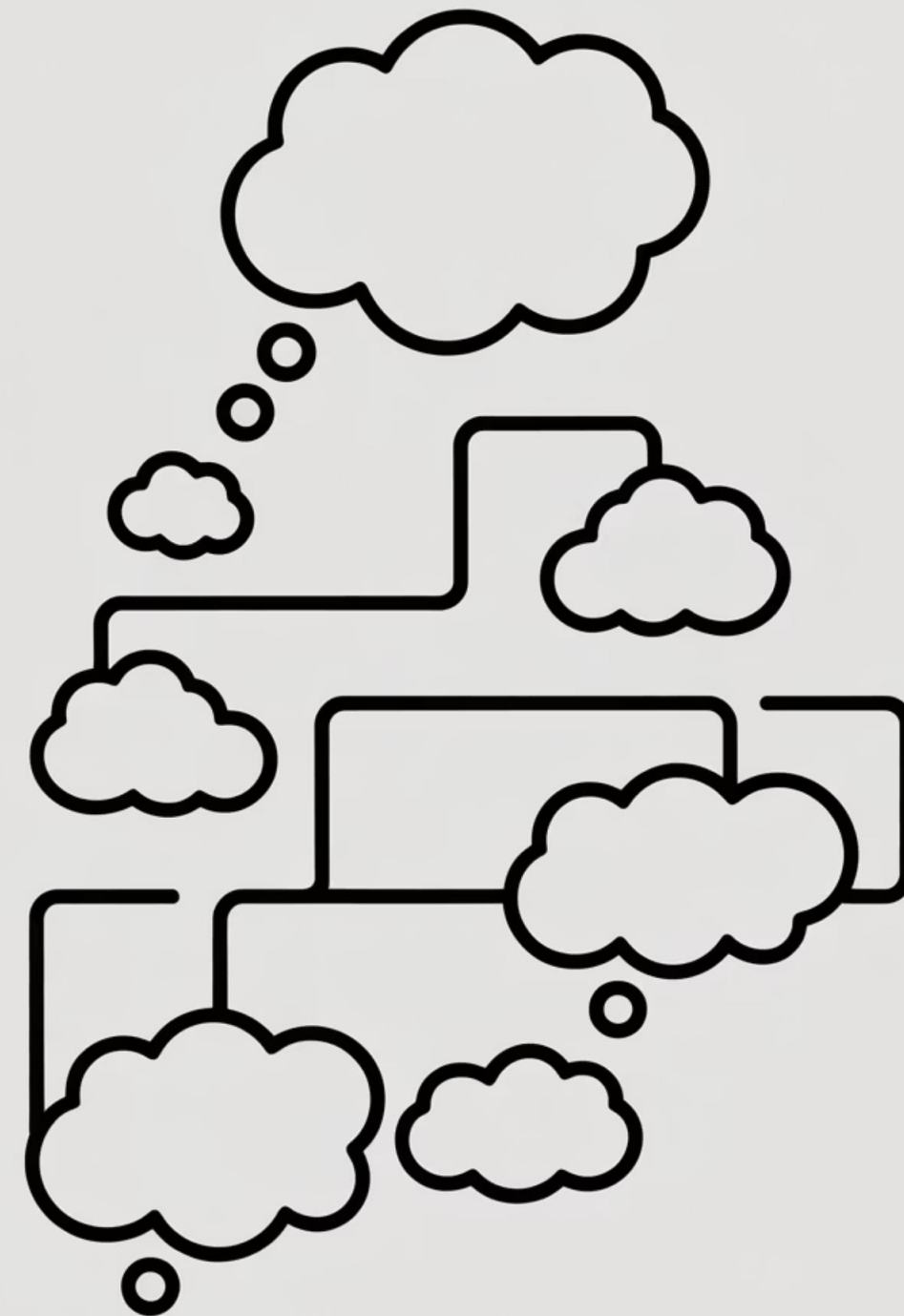
С Chain-of-Thought

Вопрос: Посчитай сумму 17, 23 и 18

Ответ: Давайте решим пошагово:

1. Сначала сложим 17 и 23: $17 + 23 = 40$
2. Теперь прибавим 18: $40 + 18 = 58$
3. Ответ: 58

Процесс прозрачен, легко проверить и исправить.



Few-Shot и Zero-Shot обучение

После instruction tuning модели приобретают удивительную способность справляться с новыми задачами, даже если не видели точно таких примеров в обучающих данных:

Zero-Shot

Модель выполняет задачу **без примеров**, только на основе инструкции.

Инструкция: "Определи тональность отзыва: положительная или отрицательная"

1

Few-Shot

Даётся **несколько примеров** (обычно 2-5) для лучшего понимания формата.

3-5 примеров различных отзывов с тональностью

2

3

One-Shot

Даётся **один пример** выполнения задачи перед основным запросом.

Пример: "Отличный продукт! → Положительная"

Промпт-инжиниринг — профессия будущего?

01

Цель (задача)

Что вы хотите получить? (ответ, текст, код, идею и т.д.)

02

Контекст

Кто вы, для кого, в какой ситуации? Пример: «Ты — маркетолог, пишешь пост для Instagram»

03

Формат ответа

Список, абзац, таблица, диалог, код и т.п.

04

Ограничения и требования

Длина, стиль (официальный, дружелюбный), запрещённые темы

05

Пример (опционально)

«Напиши как в стиле Достоевского» или «Пример: Привет! Сегодня поговорим о...»

Методология CRISP

Чёткий, Релевантный, Информативный, Структурированный, Целевой

Подходит для деловых, аналитических и образовательных задач.

Clear (Чёткий)

Избегайте двусмысленностей

Relevant (Релевантный)

По теме, без лишнего

Informative (Информативный)

Содержит нужный контекст

Structured (Структурированный)

Логичная последовательность

Purposeful (Целевой)

Понятно, зачем нужен промпт

📄 **Пример:** «Проанализируй причины падения продаж в Q2 (Clear, Relevant). Укажи 3 основные причины, подкрепи данными (Informative, Structured). Цель — презентация руководству (Purposeful)»

Методология STAR

Situation, Task, Action, Result

Используется для повествования, кейсов, описания опыта или генерации историй.



Situation

В чём суть ситуации?



Task

Какова была задача?



Action

Какие действия предприняты?



Result

К какому результату это привело?

❏ **Пример:** «Опиши ситуацию, когда ты внедрял Agile в команду (S). Какую задачу ты ставил (T)?
Какие шаги предпринял (A)? Какой результат достигнут (R)?»

Методология APE

Action, Purpose, Expectation

Простой и мощный фреймворк для быстрой формулировки промптов.

Action

Что нужно сделать?

Purpose

Зачем?

Expectation

Что ожидается в ответе?

📄 **Пример:** «Напиши (Action) краткое письмо клиенту о задержке поставки (Purpose), в вежливом тоне, не более 100 слов (Expectation)»

ROLE + TASK + FORMAT + CONSTRAINTS

Универсальный шаблон для сложных запросов: «Ты — копирайтер с 10-летним опытом» + «Напиши заголовок для рекламы кофе» + «В стиле заголовка в Forbes» + «Не более 8 слов, с эмоциональным акцентом»

Инструкции vs «простые» LLM

X – Вход

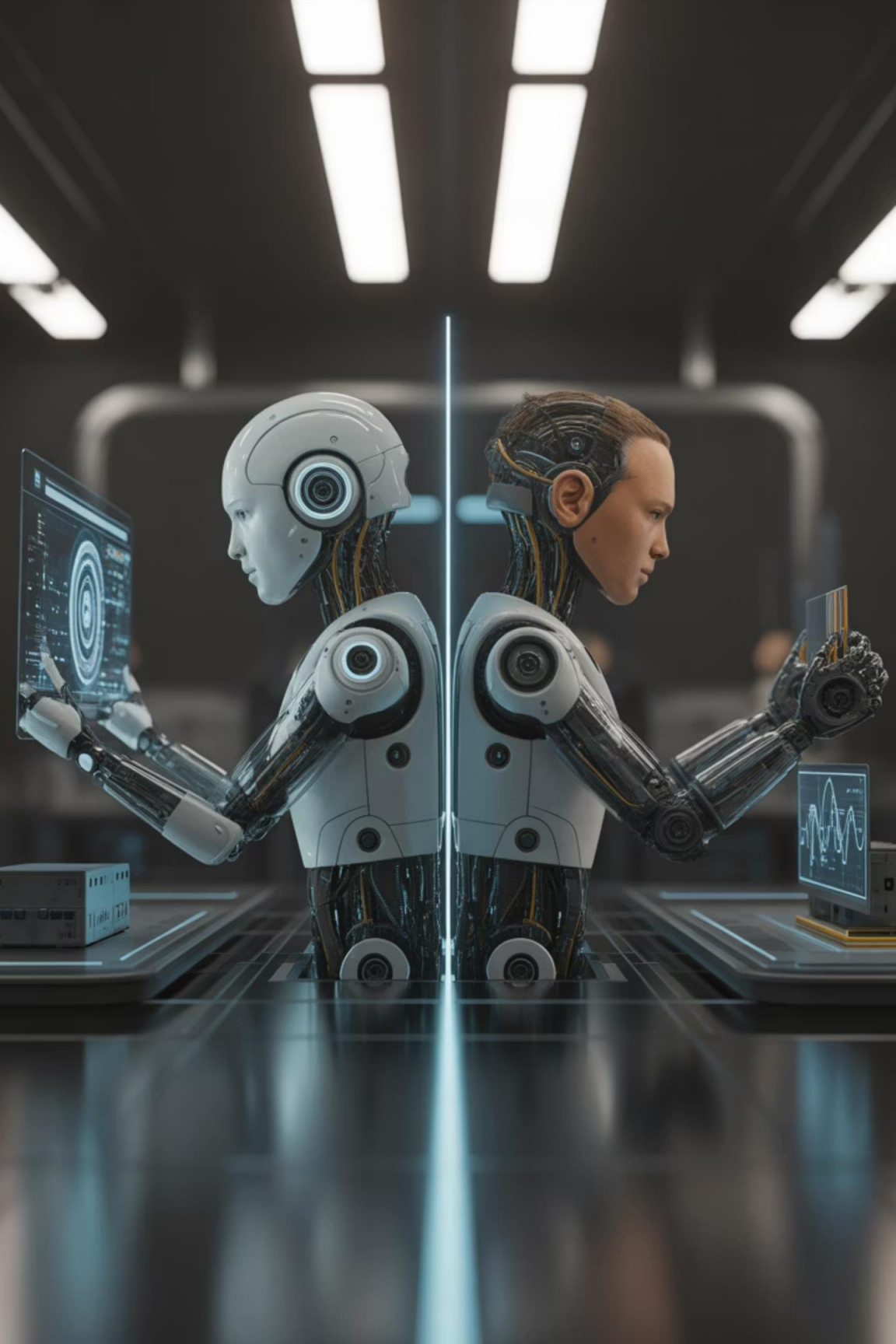
Естественный язык,
превращенный в
токены

f – Функция

Векторы Embedding,
взаимодействующие
через механизм
внимания, **при этом в
весах заложено
понимание того, как
надо выполнять
команды/инструкции**

y – Выход

Последовательность
вероятностей токенов



RAG is All You Need



Конфликт интересов

Ну и при его программировании Третий Закон был задан особенно строго... Его стремление избежать опасности необыкновенно сильно. А когда ты послал его за селеном, ты дал команду небрежно, между прочим, так что потенциал, связанный со Вторым Законом, был довольно слаб.

Около селенового озера существует какая-то опасность. Она возрастает по мере того, как робот приближается, и на каком-то расстоянии от озера потенциал Третьего Закона становится в точности равен потенциалу Второго Закона.

— Айзек Азимов, «Хоровод» (Runaround)

Четыре компонента галлюцинаций LLM

Знания и понимание

- Пробелы в обучающих данных
- Ошибочные взаимосвязи между концепциями
- Устаревшая информация в базе знаний

Недостаток контекста

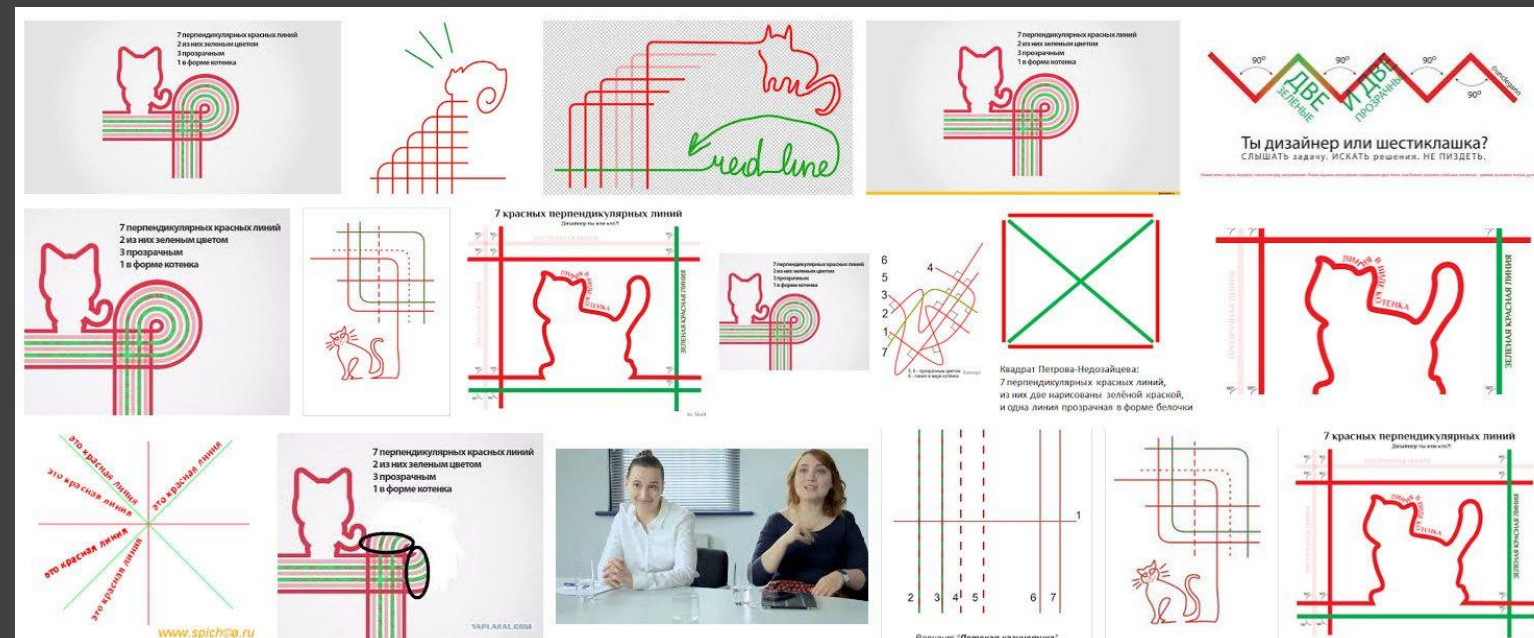
- Неполная информация о задаче
- Отсутствие релевантных деталей
- Неясные пользовательские запросы

Сложные задачи

- Многоступенчатое рассуждение
- Специализированные знания
- Противоречивые требования

Инструкции и Alignment

- Попытка угодить всем запросам
- Конфликт между безопасностью и полезностью
- Отсутствие механизма отказа



<https://spark.ru/startup/belij-muravej/blog/31272/sem-krasnih-linij>

Проблема: устаревшие знания

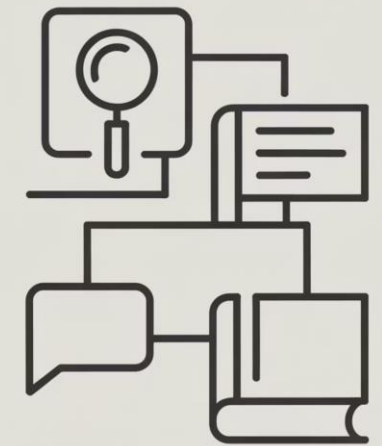
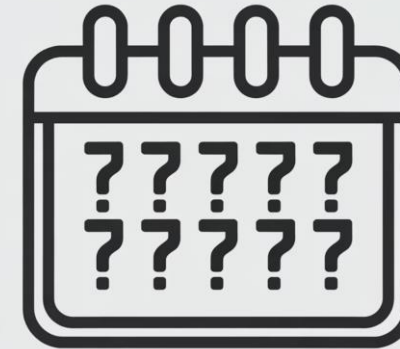
У языковых моделей есть фундаментальная проблема: их знания ограничены датой обучения. Если модель обучена на данных до 2023 года, она ничего не знает о событиях 2024-2025 годов.

Кроме того, модели не имеют доступа к специализированной информации: внутренним документам компаний, личным данным, узкоспециальным базам знаний.

Как дать модели доступ к актуальной и специализированной информации?

Решение: RAG

Retrieval Augmented Generation (RAG) — это архитектурный подход, который соединяет языковую модель с системой поиска информации. Модель получает доступ к внешним источникам знаний и использует найденную информацию для генерации ответов.



Как работает RAG: три этапа

1

Retrieval (Поиск)

По запросу пользователя система ищет релевантную информацию в базе знаний. Используется семантический поиск через эмбединги.

2

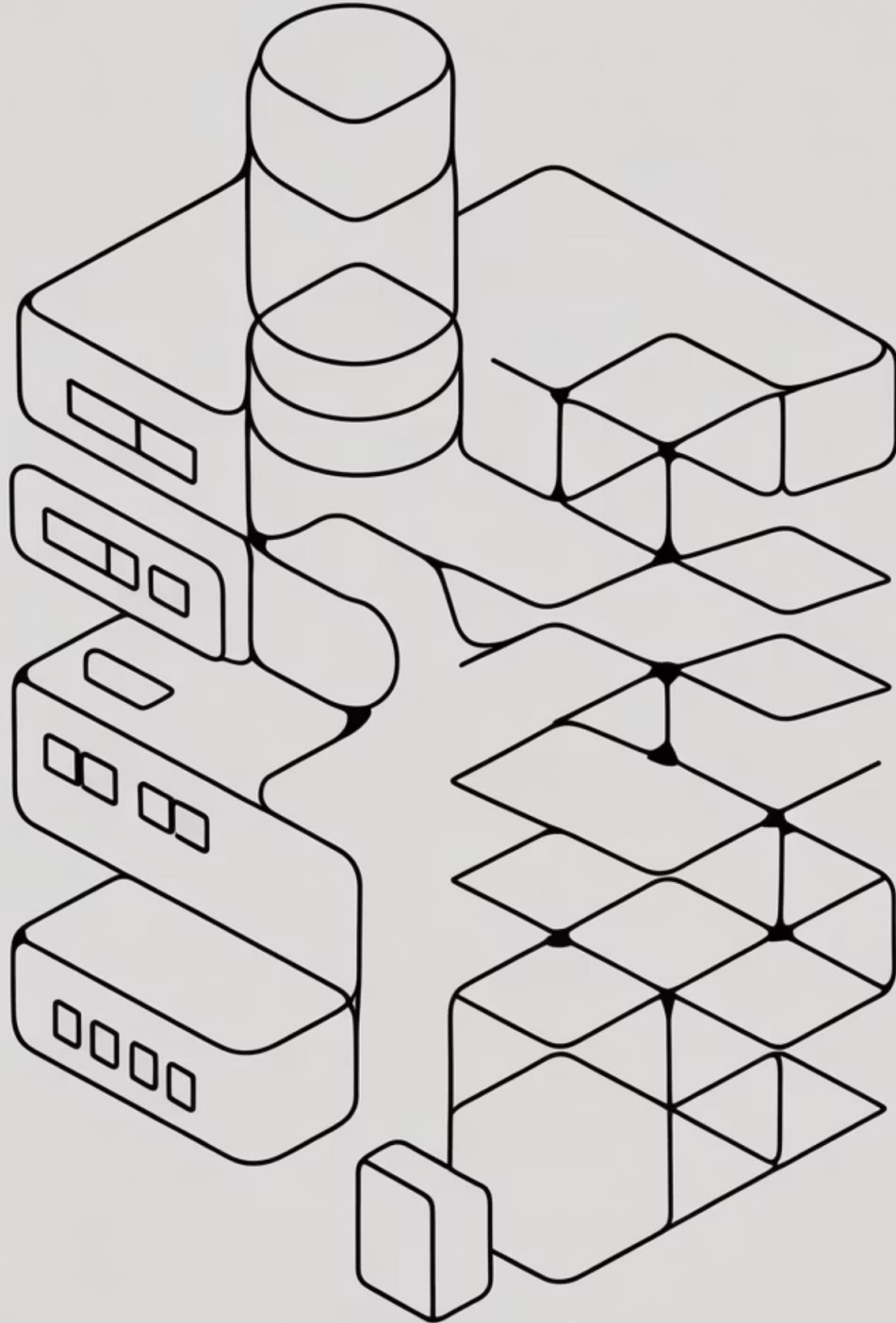
Augmentation (Дополнение)

Найденная информация добавляется к исходному запросу пользователя в качестве контекста.

3

Generation (Генерация)

Языковая модель генерирует ответ, основываясь на расширенном контексте с актуальной информацией.



Векторные базы данных

Векторные базы данных — специализированные хранилища, оптимизированные для поиска по семантическому сходству. Они умеют быстро находить векторы, наиболее близкие к запросу.



Скорость

Специальные алгоритмы (HNSW, IVF) позволяют искать среди миллионов векторов за миллисекунды.



Масштабируемость

Могут хранить миллиарды векторов и масштабироваться горизонтально.

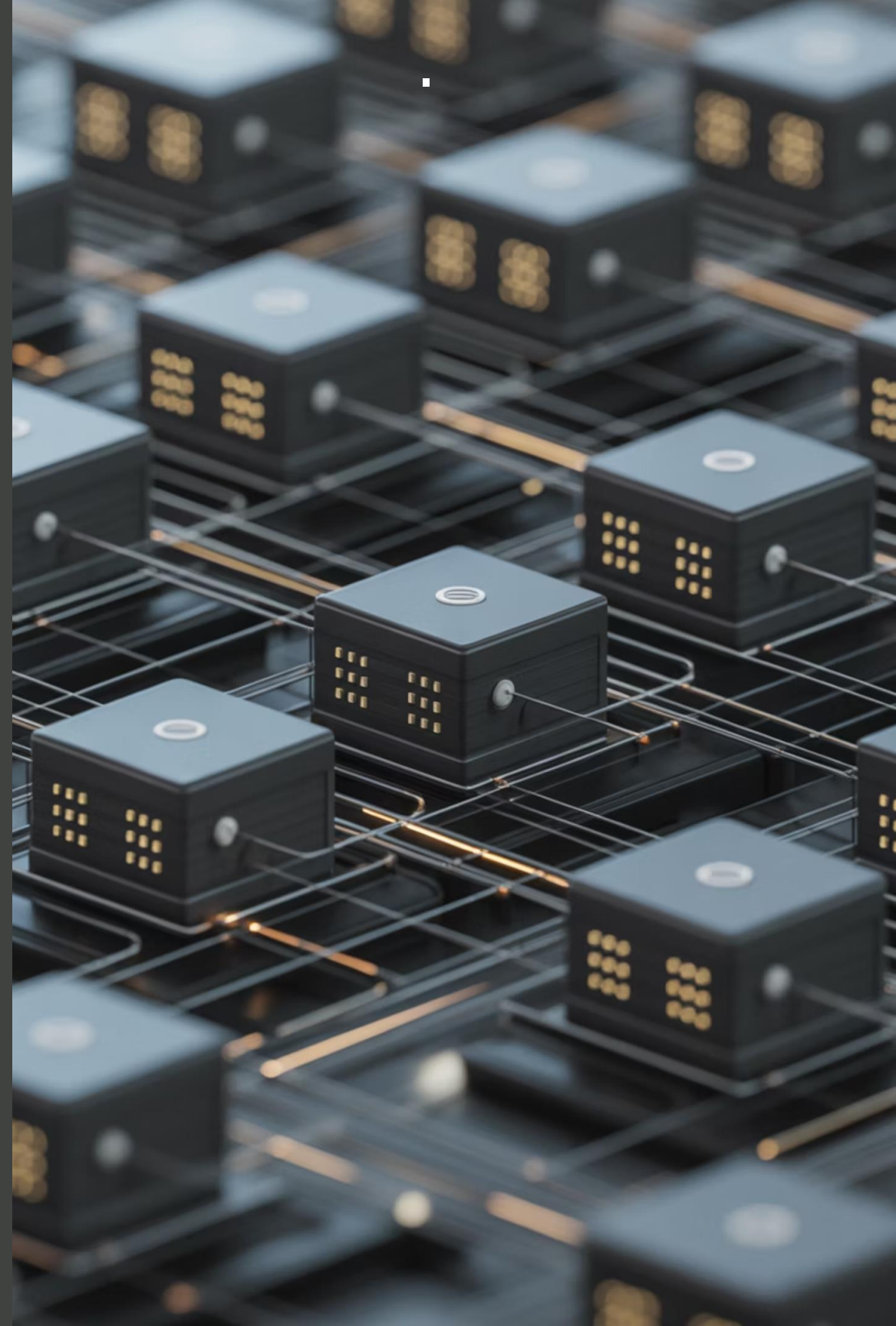


Фильтрация

Поддержка метаданных для комбинирования семантического поиска с традиционными фильтрами.

GraphRAG: тренды развития

GraphRAG представляет собой эволюцию традиционного RAG, использующую графовые структуры для более эффективного поиска и связывания информации.



Что даёт нам RAG

X – Вход

Естественный язык,
превращенный в
токены, **дополненный**
«нужным» контекстом

f – Функция

Векторы Embedding,
взаимодействующие
через механизм
внимания с «базой
фактов», при этом в
весах заложено
понимание инструкций

y – Выход

Последовательность
вероятностей токенов



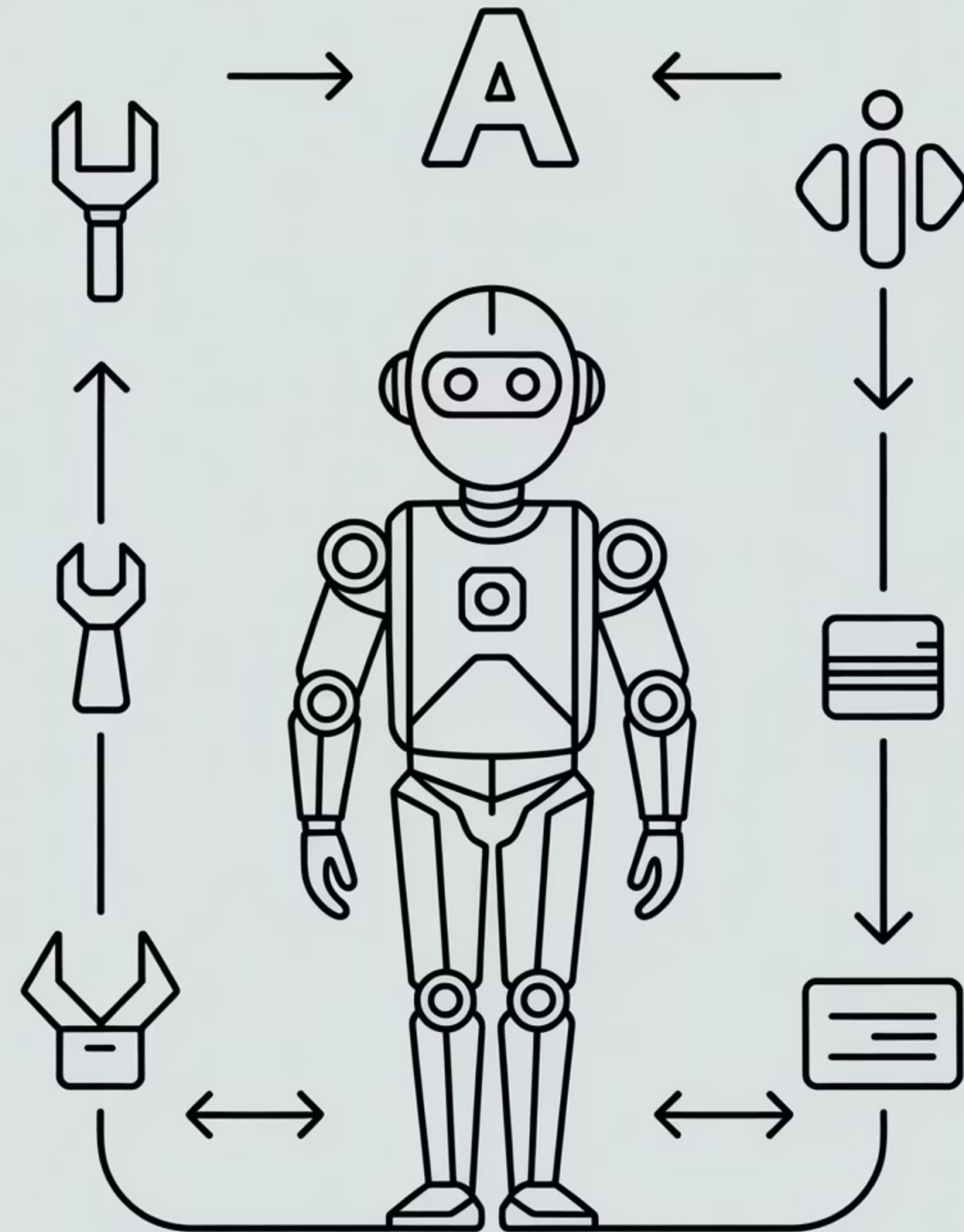
Agent is All You Need



От чат-бота к агенту

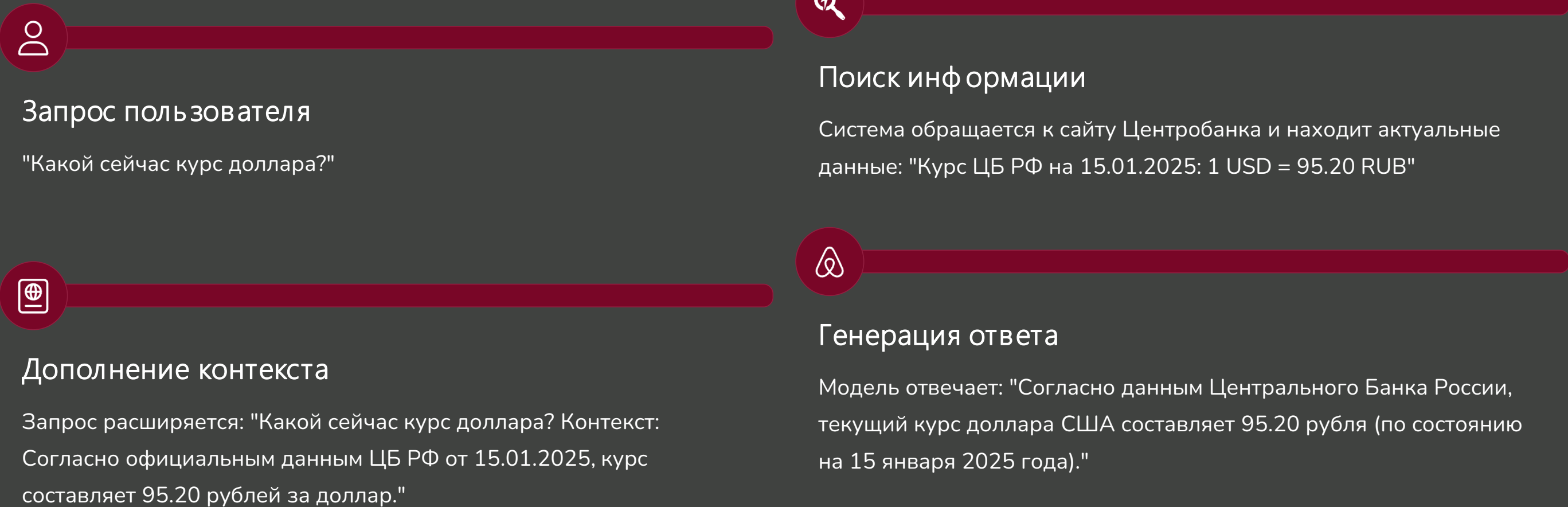
Обычный чат-бот отвечает на вопросы, но не более того. **AI-агент** — это качественно новый уровень: автономная система, которая может планировать действия, использовать инструменты, запоминать контекст и самостоятельно достигать поставленных целей.

Агент — это не просто модель, а целая система, включающая память, инструменты, логику планирования и выполнения действий.



Практический пример Агента

Рассмотрим вопрос пользователя: "Какой сейчас курс доллара?"



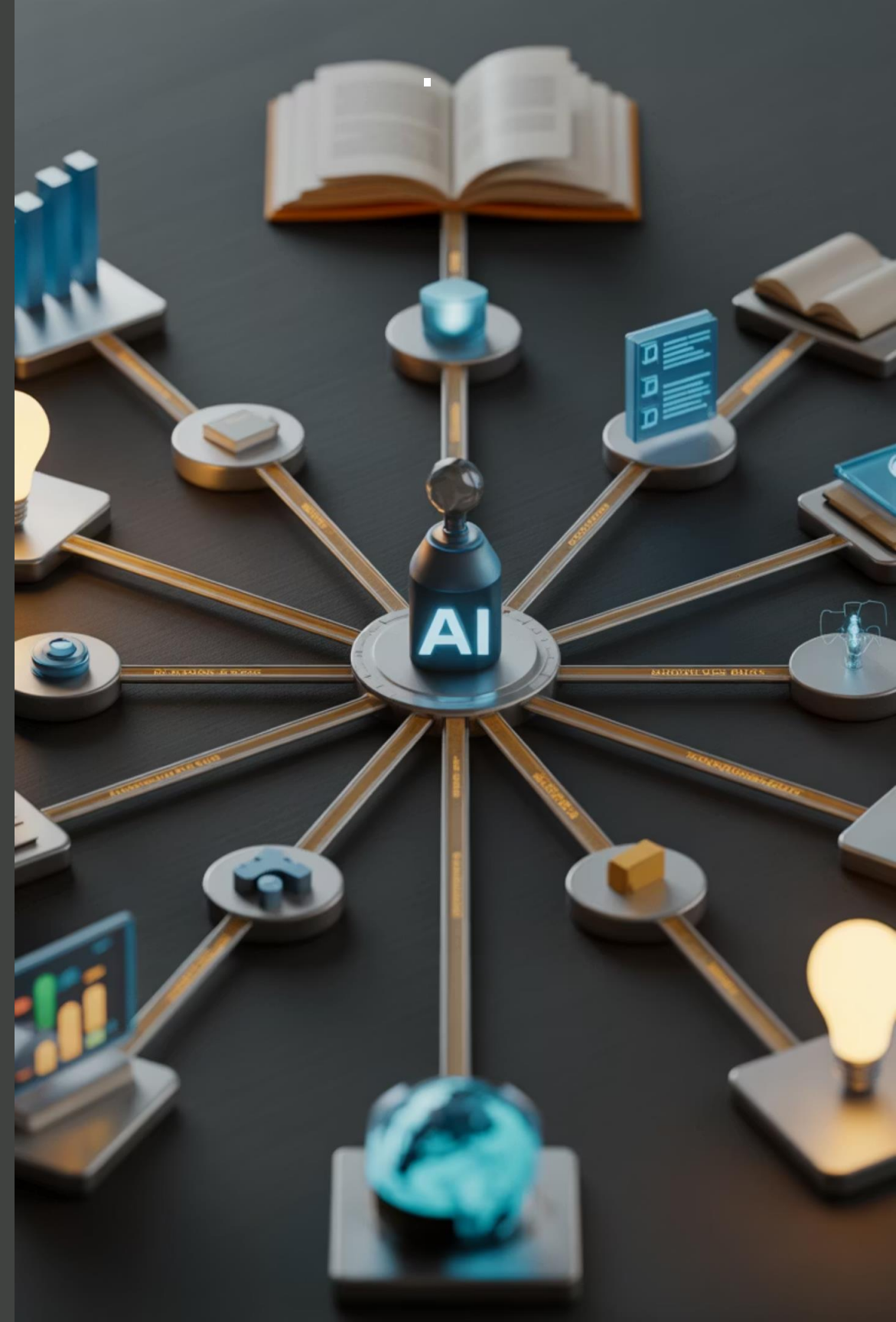
Пример работы агента: бронирование путешествия

Задача: пользователь просит "Забронируй мне отель в Париже на следующие выходные"



К чему это может привести

Агенты могут выполнять сложные многошаговые задачи, используя различные инструменты и принимая решения на каждом этапе.



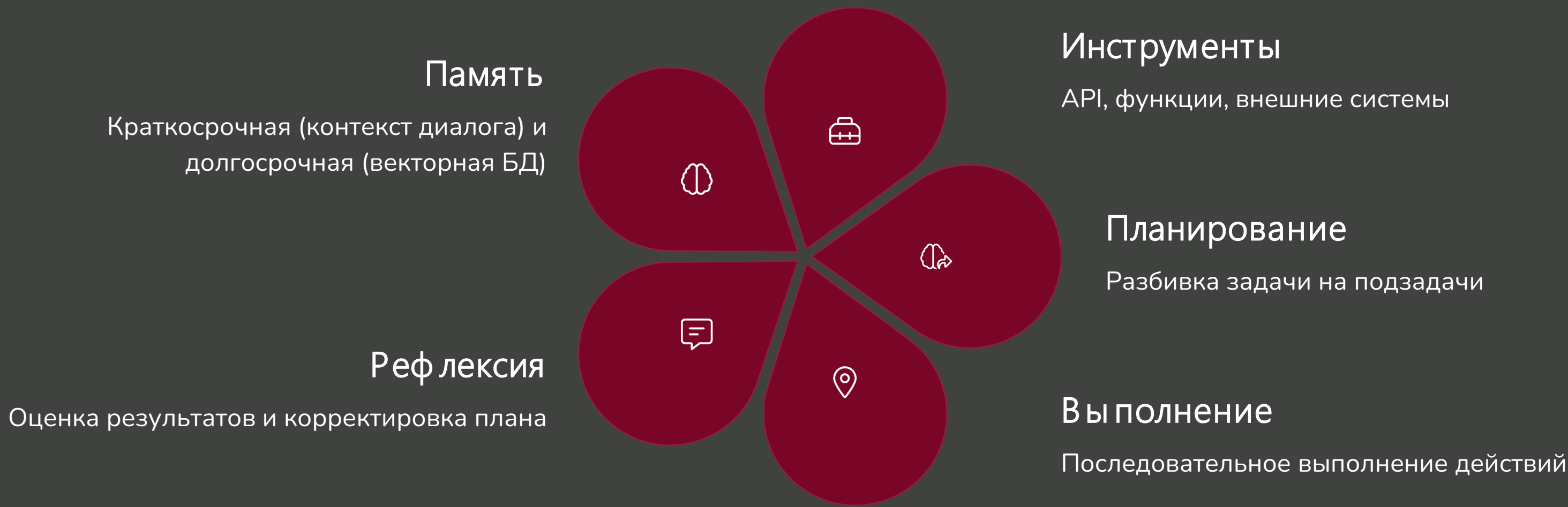
Цикл работы агента: Чувствовать → Думать → Действовать

AI-агент работает в циклической манере, постоянно воспринимая окружающую среду, принимая решения и выполняя действия:



Этот цикл повторяется до достижения цели или исчерпания ресурсов (времени, попыток, бюджета токенов).

Ключевые компоненты AI-агента



Инструменты (Tools) для агентов

Инструменты расширяют возможности агентов, позволяя им взаимодействовать с внешним миром. Каждый инструмент — это функция с описанием, которую агент может вызвать.



Веб-поиск

Поиск актуальной информации в интернете через Google, Bing или специализированные API.



Базы данных

Выполнение SQL-запросов для получения или изменения структурированных данных.



Выполнение кода

Запуск Python, JavaScript или другого кода для вычислений, анализа данных, визуализации.



Внешние API

Взаимодействие с любыми веб-сервисами: отправка email, создание задач, управление календарём.



Калькулятор

Точные математические вычисления без ошибок округления или галлюцинаций модели.



Работа с файлами

Чтение, запись, парсинг различных форматов файлов: PDF, Excel, CSV, JSON.

Мультиагентные системы

Вместо одного универсального агента можно создать **команду специализированных агентов**, где каждый эксперт в своей области. Это приводит к более качественным и масштабируемым решениям.



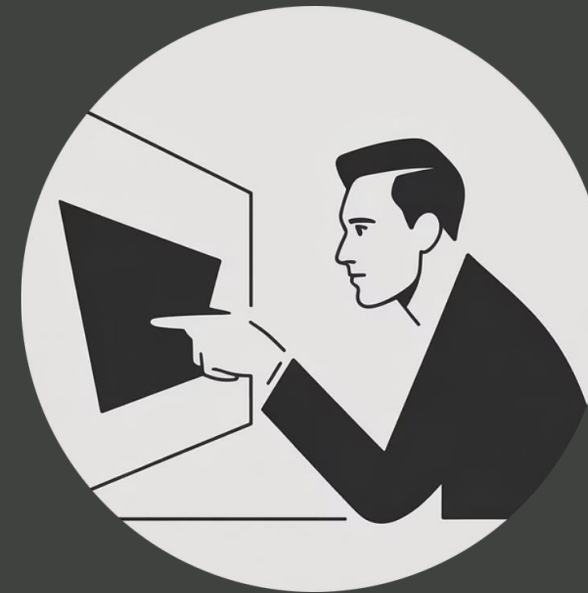
Агент-исследователь

Собирает и анализирует информацию из различных источников. Специализируется на поиске и верификации фактов.



Агент-писатель

Создаёт структурированный контент на основе собранной информации. Владеет разными стилями письма.



Агент-критик

Проверяет качество работы других агентов, находит ошибки, предлагает улучшения.



Агент-координатор

Управляет потоком задач между агентами, следит за прогрессом, принимает решения о приоритетах.

Ограничения и вызовы агентов

Несмотря на впечатляющие возможности, AI-агенты сталкиваются с рядом серьёзных проблем:

Технические ограничения

- **Галлюцинации:** агент может выдумывать факты или действия
- **Стоимость:** каждое действие требует вызова API модели
- **Латентность:** сложные задачи требуют много итераций
- **Зацикливание:** агент может застрять в повторяющихся действиях
- **Ошибки инструментов:** API могут падать или возвращать неожиданные результаты

Проблемы безопасности

- **Prompt injection:** злоумышленник может манипулировать инструкциями агента
- **Неконтролируемые действия:** агент может выполнить нежелательные операции
- **Утечка данных:** агент может случайно раскрыть конфиденциальную информацию
- **Эскалация привилегий:** агент получает доступ к ресурсам, к которым не должен

Промпт-инжиниринг мертв?

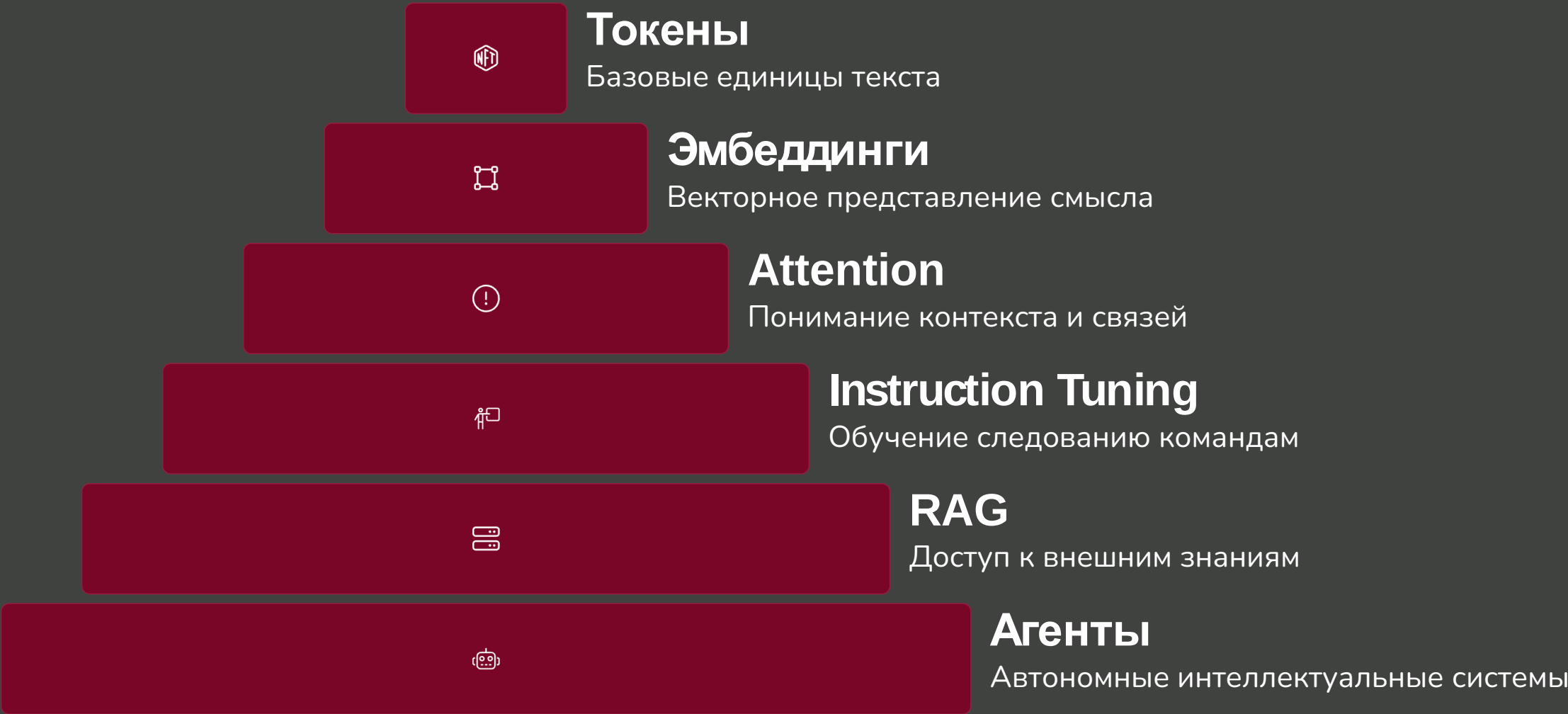


OpenAI выпустили свой генератор промтов, который автоматически оптимизирует запросы пользователей.

Означает ли это конец промт-инжиниринга как профессии? Или это просто следующий уровень абстракции?

От токенов до агентов: единая экосистема

Все рассмотренные технологии образуют единую, взаимосвязанную экосистему современного искусственного интеллекта. Каждый уровень строится на предыдущем:



Будущее развития языковых моделей

Индустрия AI развивается стремительно.
Вот ключевые направления, которые будут определять ближайшие годы:



Резюме: Everything is All You Need

1

Token is All You Need

Единица информации для LLM не всегда слово ~ 3/4 слова

2

Embedding is All You Need

Токены для LLM это векторы в очень большом пространстве

3

Attention is All You Need

Считаем «хитрую» корреляцию между всеми векторными представлениями в предложении и смешиваем их с «фактами»

4

Instruction is All You Need

Не просто учимся генерировать следующий токен, а следуем инструкции

5

RAG is All You Need

Сохраняем текст в векторе, чтобы наиболее точно добавить в промпт

6

Agent is All You Need

Структурный подход к запросам. Добавляем в модель рефлекссию и не LLM-ные функции для лучших ответов

При подготовке презентации использовались

- 1 Долганов Антон Юрьевич**
Была обычная презентация сделанная руками (с примерами из Qwen)
- 2 Perplexity AI**
Для формирования «обзорного текста»
- 3 Gamma.app**
Для генерации нескольких черновых презентаций в общей концепции
- 4 Долганов Антон Юрьевич**
Который всё это сводил

Спасибо за внимание!

RU Русский

Есть ли вопросы?

Буду рад ответить на ваши вопросы

US English

Any questions?

I'd be happy to answer your questions

FR Français

Des questions?

Je serais ravi de répondre à vos questions

DE Deutsch

Fragen?

Ich beantworte gerne Ihre Fragen

ES Español

¿Preguntas?

Estaré encantado de responder sus preguntas

CN 中文

有问题吗?

我很乐意回答您的问题

Готов к диалогу

Ваши вопросы помогут нам углубить понимание темы и найти новые точки зрения

Благодарю за ваше время и внимание! Жду ваших вопросов и комментариев.